

Topics

Sunday, July 20, 2025

10:08 AM

- 1. Introduction to DSA and Python Basics**
- 2. Arrays and Lists**
- 3. Strings**
- 4. Linked Lists**
- 5. Stacks**
- 6. Queues**
- 7. Searching**
- 8. Sorting**
- 9. Recursion**
- 10. Hashing**
- 11. Patterns & Problem Solving**
- 12. Trees**
- 13. Graphs**

1 - Introduction

Saturday, July 26, 2025 9:29 AM

- **Importance of DSA**
- **Time & Space Complexity**
- **Python Refresher**

2. Arrays & Lists

Saturday, July 26, 2025

5:10 PM

- **Arrays**
- **Lists**
- **Arrays vs Lists**
- **Complexity Analysis**
- **Coding Problems**

3. Strings

Saturday, August 16, 2025 4:13 PM

- **Introduction**
- **Properties**
- **Usecases**
- **Common Functions**
- **String Manipulation Techniques**
- **String Comparison**
- **Coding Problems**

4. Linked Lists

Wednesday, August 27, 2025

1:55 PM

- **Introduction**
- **Linked Lists in Memory**
- **Linked Lists vs Arrays/Lists**
- **Types of Linked Lists**
- **Singly Linked List**
- **Circular Singly Linked List**
- **Doubly Linked List**
- **Circular Doubly Linked List**
- **Coding Problems**

5. Stacks

Saturday, October 18, 2025

11:24 AM

- **Introduction**
- **Common Operations**
- **Implementation**
- **Complexity Analysis**
- **Applications**
- **Pros & Cons**
- **Coding Problems**

6. Queues

Saturday, October 18, 2025 3:01 PM

- **Introduction**
- **Common Operations**
- **Implementation**
- **Complexity Analysis**
- **Coding Problems**

7. Searching

Wednesday, October 29, 2025

10:09 PM

- **Linear Search**
- **Binary Search**
- **Exponential Search**

8. Sorting

Wednesday, November 5, 2025

10:11 PM

- **Introduction**
- **Bubble Sort**
- **Selection Sort**
- **Insertion Sort**
- **Merge Sort**
- **Quick Sort**
- **Counting Sort**
- **Complexity Analysis**

9. Recursion

Sunday, November 23, 2025 5:51 PM

- **Introduction to Recursion**
- **Internal Working of Recursion**
- **Common Mistakes**
- **Recursion vs Iteration**
- **Coding Problems**

10 - Hashing

Wednesday, November 26, 2025

11:51 PM

- **Introduction**
- **Hash Functions**
- **Hash Tables**
- **Hash Collisions & Collision Resolution**
- **Load Factor**
- **Common Operations**
- **Complexity Analysis**
- **Coding Problems**

11 - Problem Solving

Thursday, December 4, 2025 9:17 AM

- **Sliding Window**
- **Two Pointers**
- **Fast & Slow Pointers**
- **Prefix Sum**
- **Hashing**
- **Binary Search Pattern**
- **Greedy Approach**

12 - Trees

Saturday, December 20, 2025

12:09 AM

- **Introduction**
- **Common Terminology**
- **Properties of Trees**
- **Traversal Algorithms**
- **Binary Search Trees**
- **N-ary Trees**
- **Dynamic Programming in Trees**
- **Coding Problems**

12.1 - Introduction

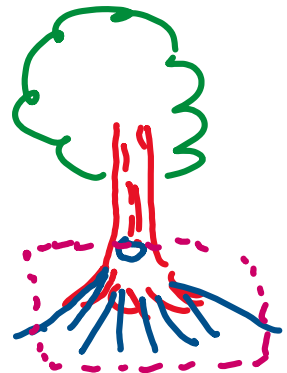
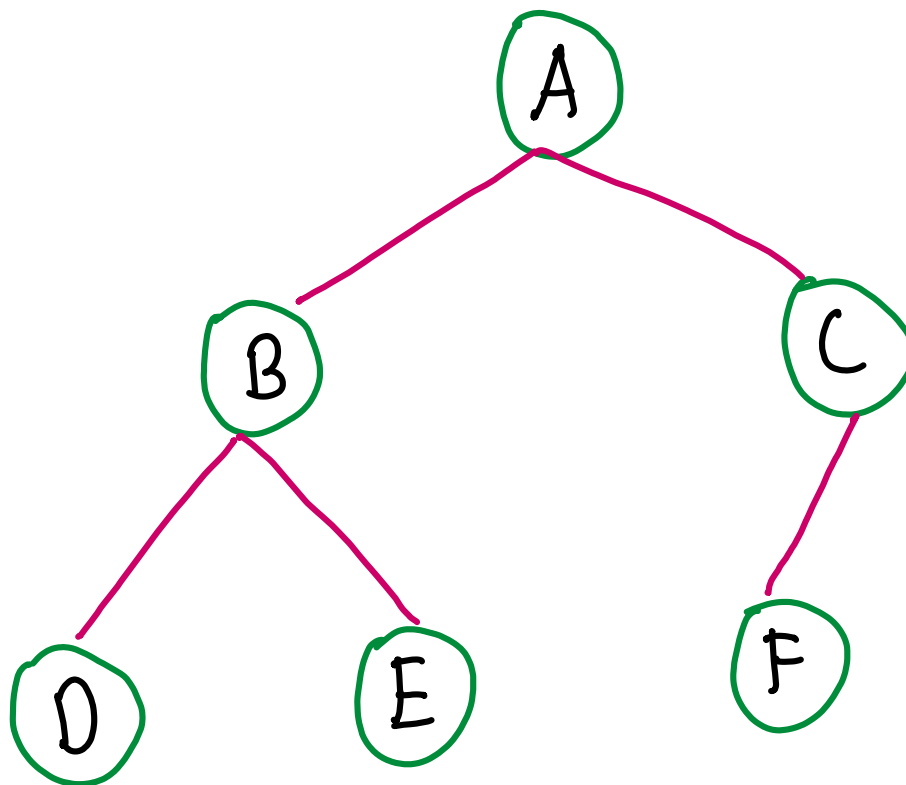
Saturday, December 20, 2025

12:18 AM

What is a Tree?

- A tree is a non-linear hierarchical data structure
- Consists of nodes connected by edges
- Data is organized in a parent-child relationship
- A tree consists of:
 - A finite set of nodes
 - Connected by edges
 - With a single root node
 - And no cycles

non-linear
hierarchical



✓ Nodes: A, B, C, D, E, F

Edges: _____

✓

✓ Edge: _____

✓ Root Node: A

12.2 - Terminology

Saturday, December 20, 2025 12:21 AM

Tree Terminology:

TERM	MEANING
Root	Topmost node
Parent	Immediate predecessor
Child	Immediate successor
Leaf	Node with no children
Depth	Distance from root (edges count)
Height	Longest path to a leaf
Level	Depth + 1
Subtree	Tree formed from any node

1. Root:

- The topmost node of the tree
- Has no parent

2. Parent:

- A node that has one or more children
- Immediate predecessor of a node

3. Child:

- A node that is a direct descendant of another node

4. Leaf:

- A node with no children
- Also called external node

5. Depth:

- Number of edges from the root to the node
- Root always has depth = 0
- Depth answers "*how far am I from root?*"

6. Height:

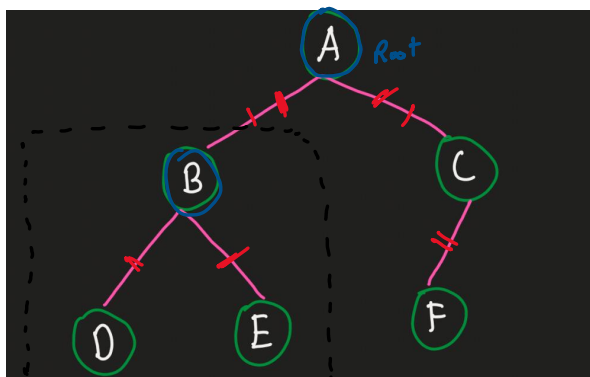
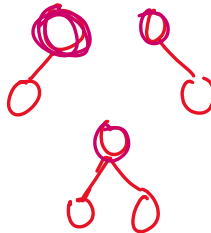
- Longest path from a node to a leaf
- Measured in edges
- Height of a leaf node = 0
- Height answers "*how far can I go down?*"

7. Level:

- Is a mathematical representation
- **Depth + 1**

8. Subtree:

- Any node along with all its descendants
- Every node forms a subtree
- Crucial in **recursion-based** tree problems



Node	Depth	Height	Level
A	0	2	1
B	1	1	2
C	1	1	2
D	2	0	3
E	2	0	3
F	2	0	3



E	2	0	3
F	2	0	3

Parent: A, B, C

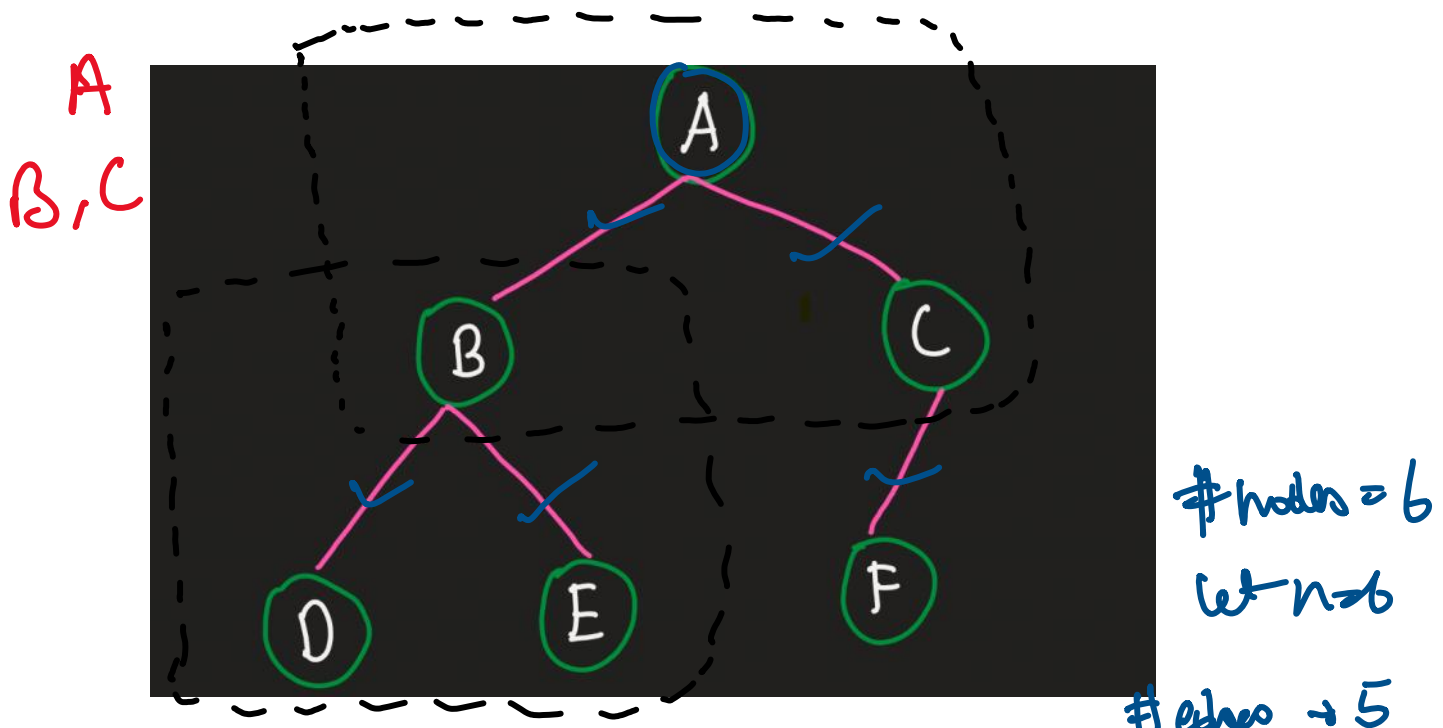
Leaf: D, E, F

12.3 - Properties

Saturday, December 20, 2025 12:20 AM

Key Properties of a Tree:

- Exactly One Root:
 - The root is the starting point of the tree
 - All nodes originate from this root
- Parent-Child Relationship:
 - Every node (except root) has exactly one parent
 - A node can have zero or more children
- No Cycles:
 - You can never return to a node once you move down the tree
 - This property distinguishes trees from graphs
- Connected Structure:
 - Every node is reachable from the root
- Edges Count Rule:
 - If a tree has n nodes, it will have exactly $(n - 1)$ edges





edges $\rightarrow 5$

12.4 - Traversal

Saturday, December 20, 2025 2:13 AM

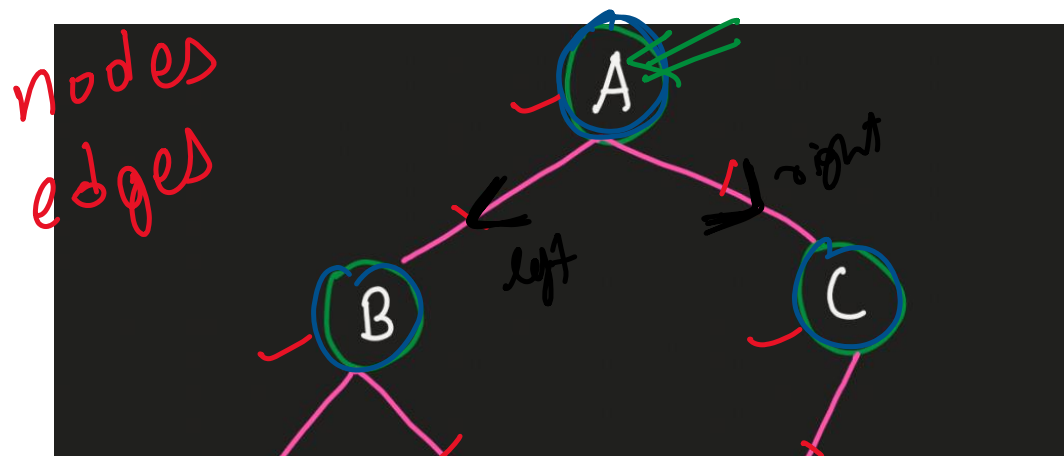
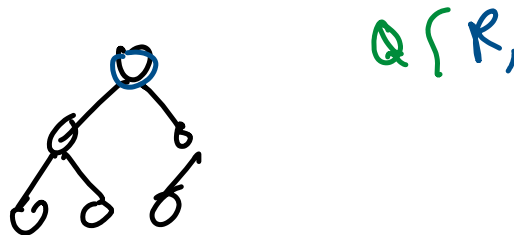
What is Tree Traversal?

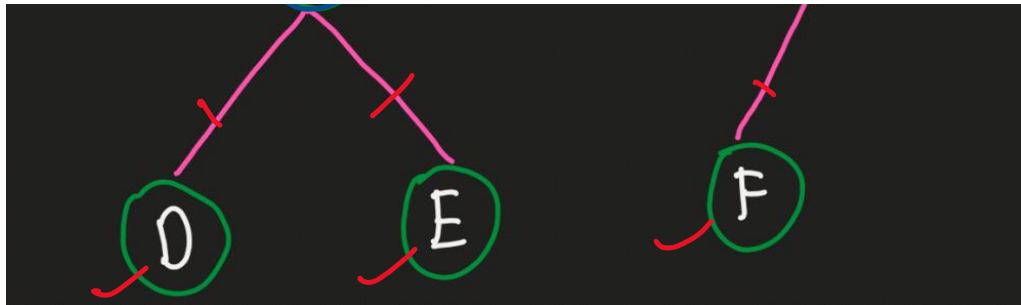
- Tree traversal is the process of visiting every node of a tree exactly once in a systematic way
- Since a tree is non-linear, there is no single natural order like arrays or linked lists
- We define multiple traversal strategies depending on problem requirements
- **Traversal answers:**
 - How do we visit every node exactly once?
 - In what order should nodes be processed?

Types of Tree Traversals:

Tree traversals are broadly classified into two categories:-

1. **Depth First Search (DFS):** *uses recursion or stack; explores one branch completely before moving to another*
 - Preorder
 - Inorder
 - Postorder
2. **Breadth First Search (BFS):** *uses a queue; explores each node level by level*
 - Level Order Traversal





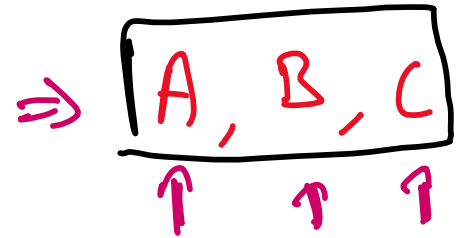
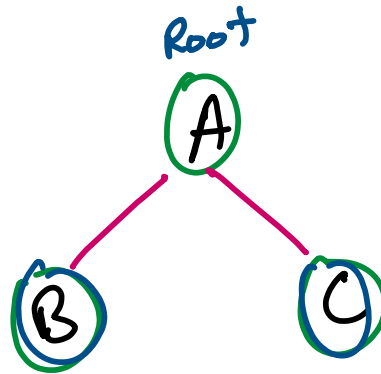
yes,
right

12.4.1 - Preorder

Saturday, December 20, 2025 11:42 AM

Order of Traversal:

Root → Left → Right

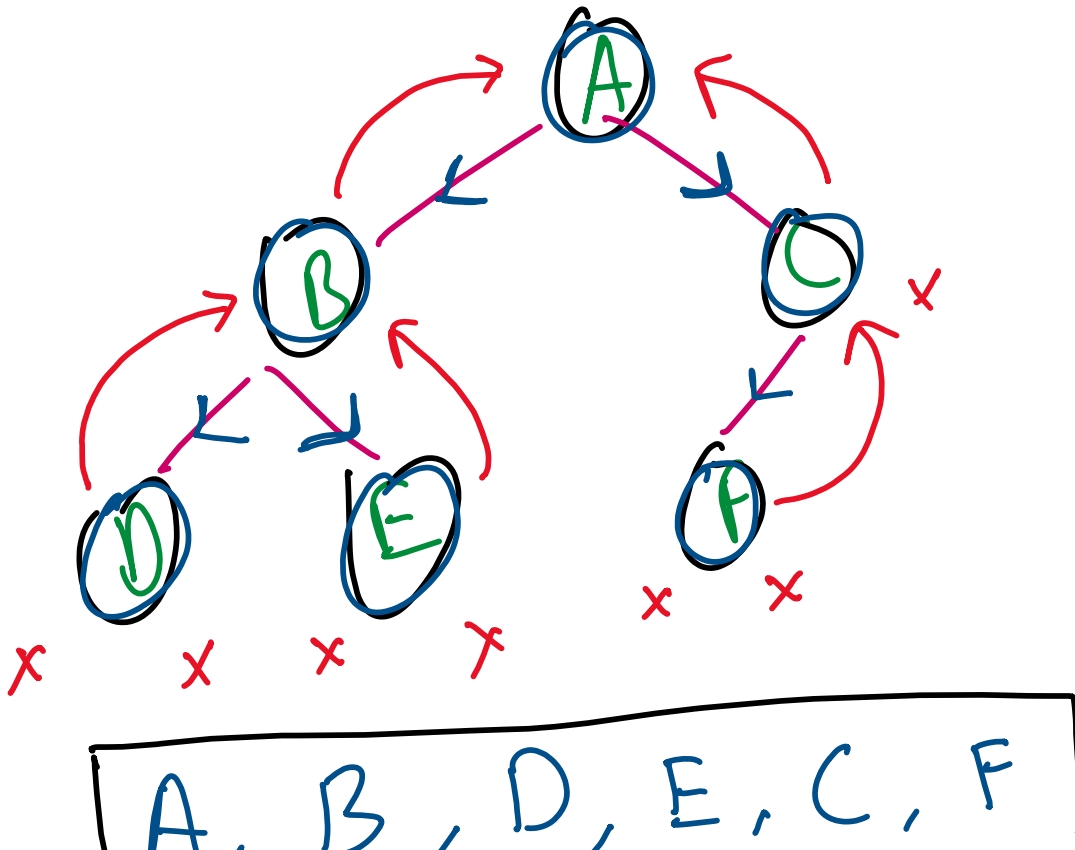


Intuition:

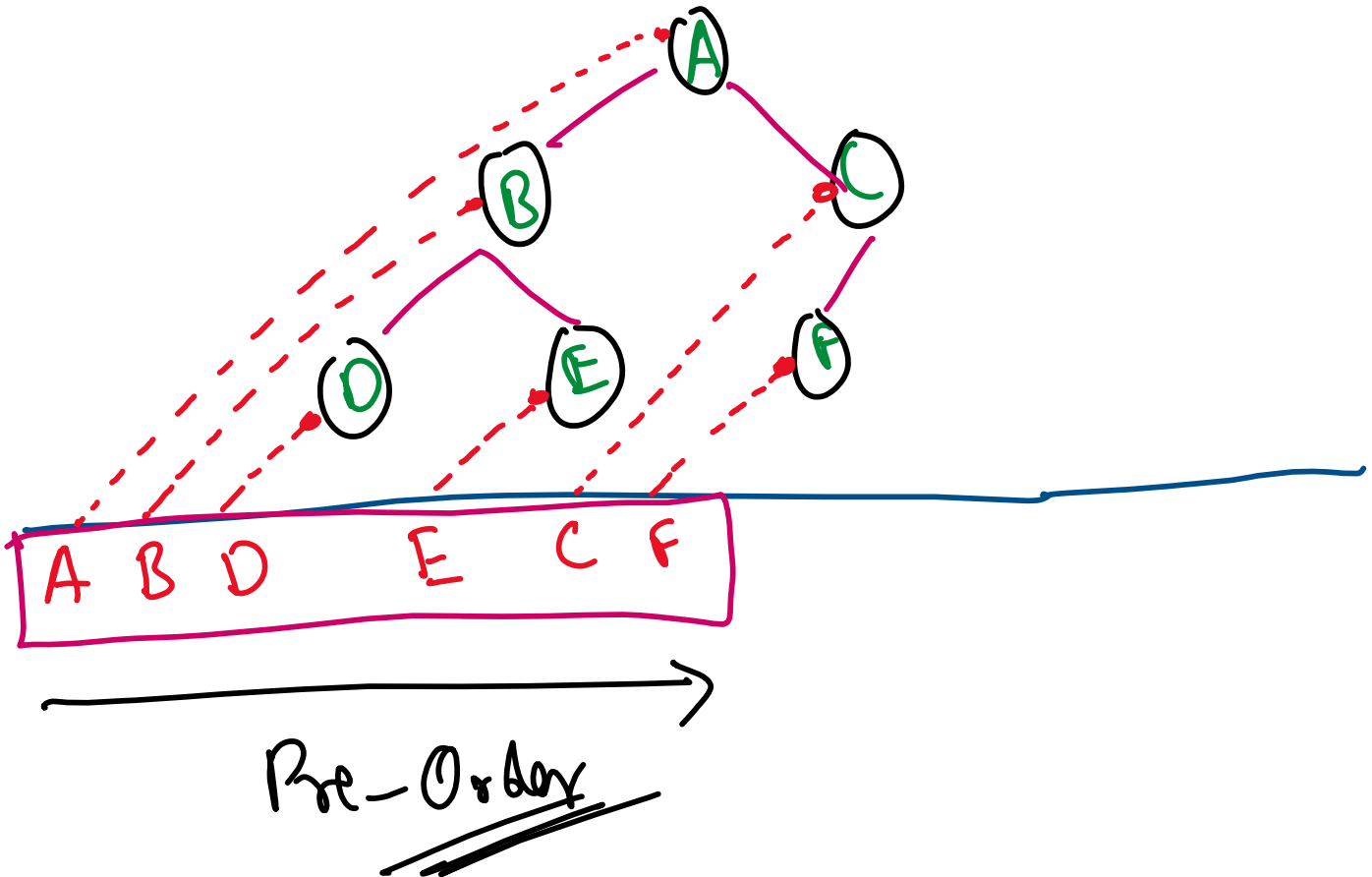
- Visit a node before its children
- Process current node, then go to left node, then right node

Usecases:

- Copying / cloning a tree
- **Prefix expression evaluation:** Operators appear before operands
- **Serialize a tree:** Convert a tree into a string or list
- **Creating directory structures:** Representing folders and files as a tree



A, B, D, E, C, F

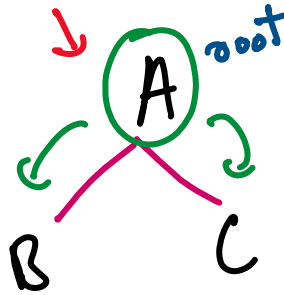


12.4.2 - Inorder

Saturday, December 20, 2025 11:50 AM

Order of Traversal:

Left → Root → Right



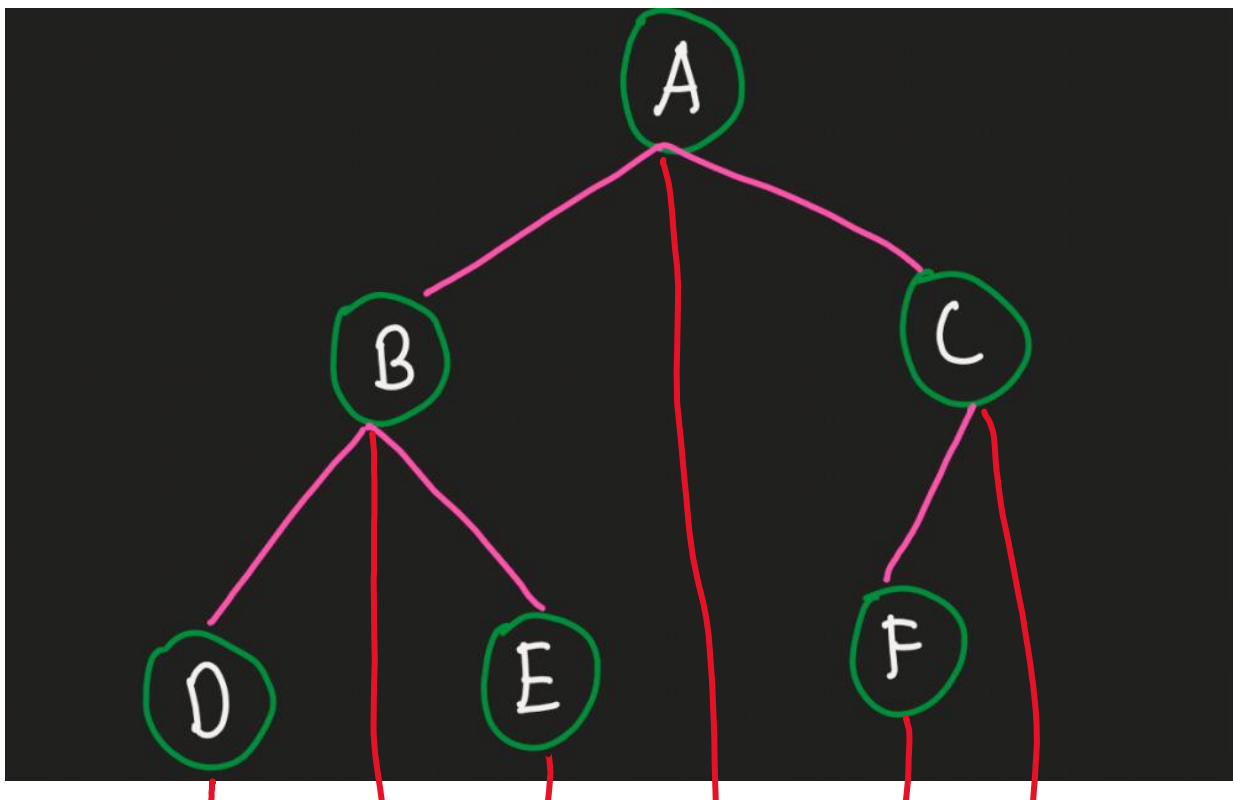
B, A, C

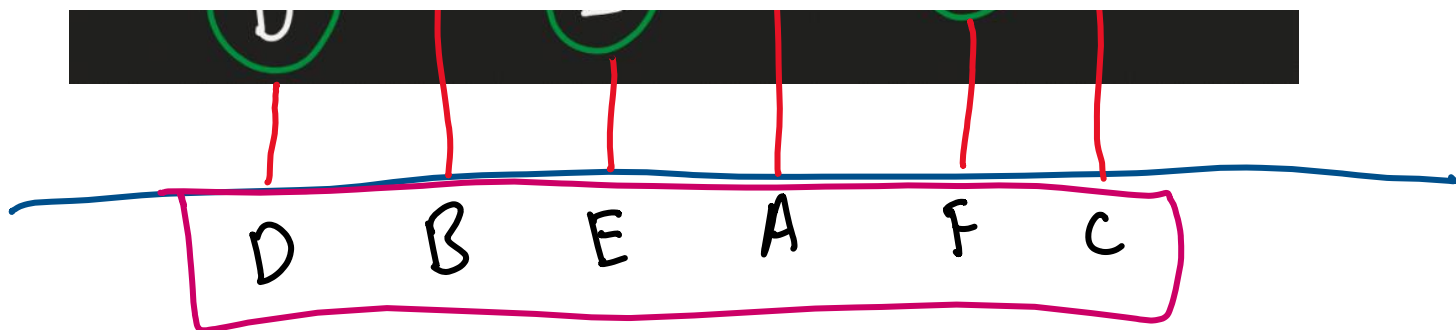
Intuition:

- Visit left child first
- Root comes in between
- Then visit right child

Use Cases:

- Validate a BST
- Retrieve sorted data from BST
- Expression trees (infix notation)
- Finding kth smallest / largest



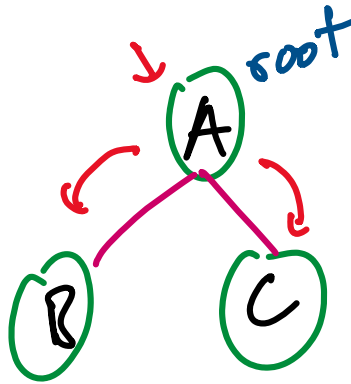


12.4.3 - Postorder

Saturday, December 20, 2025 11:51 AM

Order of Traversal:

Left → Right → Root



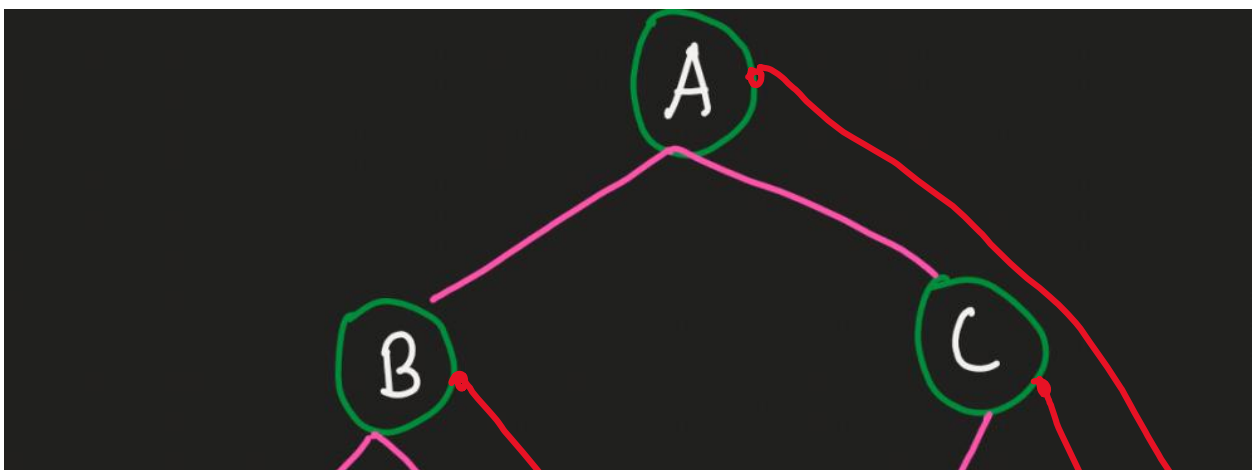
B, C, A

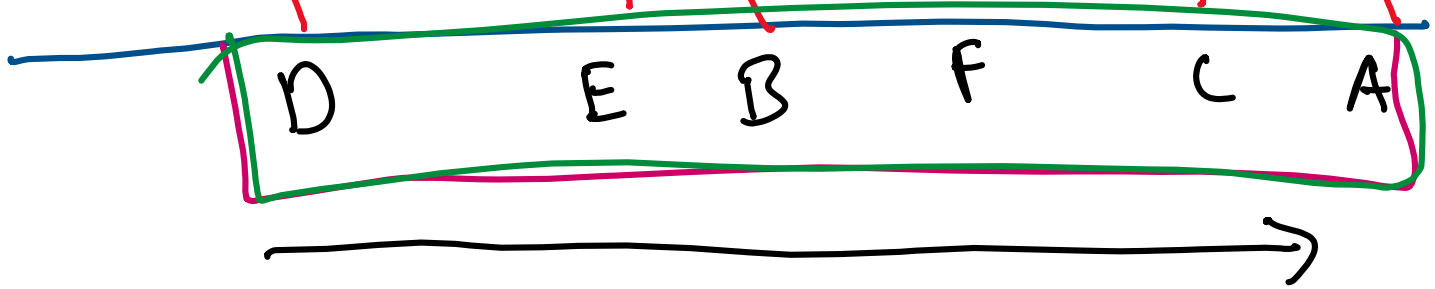
Intuition:

- Process children before parent
- Root is visited last
- Left node first, followed by right node

Use Cases:

- Deleting a tree
- Calculating height
- Postfix expression evaluation
- Bottom-up computations (DP on trees)





12.4.4 - Level Order

Saturday, December 20, 2025 11:52 AM

Order of Traversal:

- Visits nodes level by level
- Uses a queue (FIFO)

Intuition:

- Visit all nodes at depth 0
- Then depth 1
- Then depth 2
- Repeat until tree depth is reached

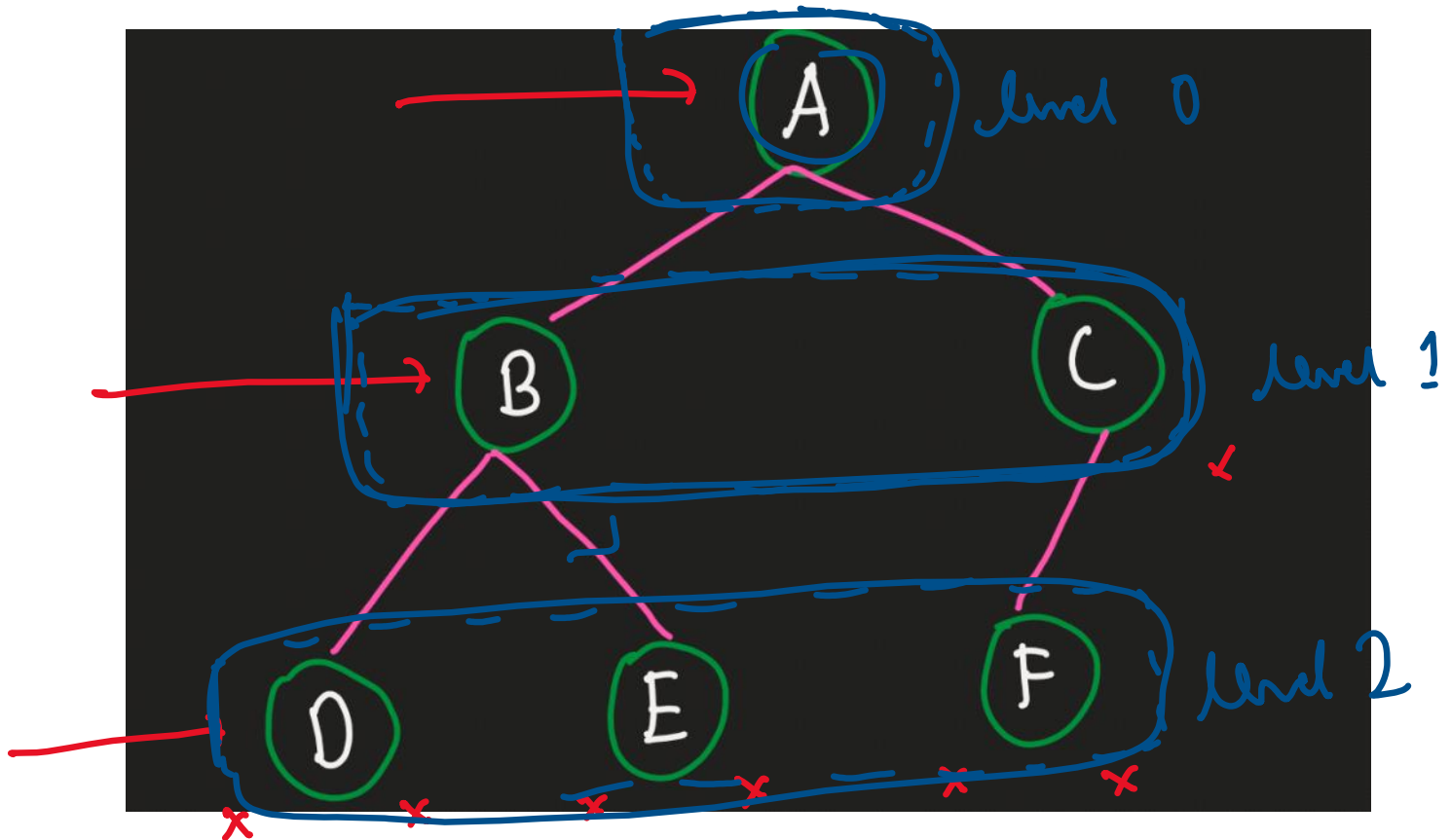
Why use a Queue?

- Must visit all nodes at level **L** before moving to level **L+1**
- Insert root into queue
- Remove (dequeue) the oldest node
- Add its children at the end of the queue
- Parents enter the queue before children

Use Cases:

- Level-based problems
- Tree views (top, bottom, left, right)
- Shortest distance in trees

- Checking tree completeness
- Zig-zag traversal



A, B, C, D, E, F

Q = ~~A~~ ~~B~~ ~~C~~ ~~D~~ ~~E~~ ~~F~~] []

Order → A B C D E F

Char $\rightarrow$ A B C D E F

A B C D E F

L0

L1

L2

12.4.5 - Choose Traversal

Saturday, December 20, 2025

12:09 PM

How to Choose the Right Traversal?

PROBLEM	TRAVERSAL
Sorted output	Inorder (BST) ✓
Root-first logic	Preorder ✓
Child-first logic	Postorder ✓
Level-wise	Level Order ✓
Height / diameter	Postorder ✓
Views / distance	Level Order ✓

12.5 - BST

Saturday, December 20, 2025 12:15 PM

What is a Binary Search Tree?

A Binary Search Tree (BST) is a binary tree with a special ordering property that enables efficient searching, insertion, and deletion

Key Properties:

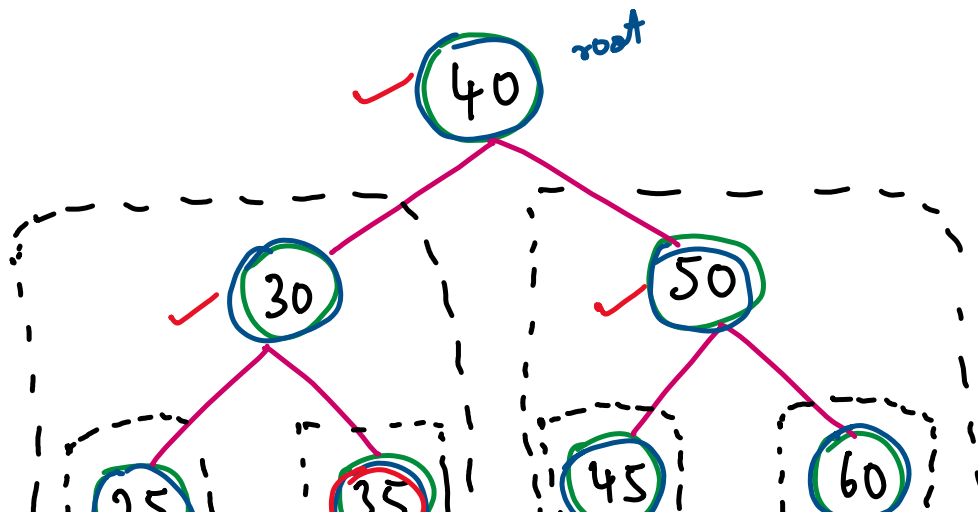
- Each node has zero, one, or two children
- All values in the left subtree are less than the node's value and all values in the right subtree are greater than the node's value
- Both, the left and right subtrees must also be Binary Search Trees themselves

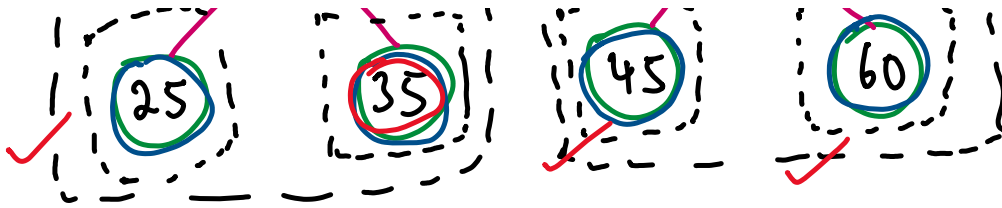
Why use a BST?

OPERATION	TIME COMPLEXITY
Search	$O(\log n)$ ✓
Insert	$O(\log n)$ ✓
Delete	$O(\log n)$ ✓

Key Applications:

- Database Indexing
- Dictionary and Spell Checkers
- Dynamic Sorting
- Priority Queues
- 3D Graphics and Game Engines





12.5.1 - Insertion

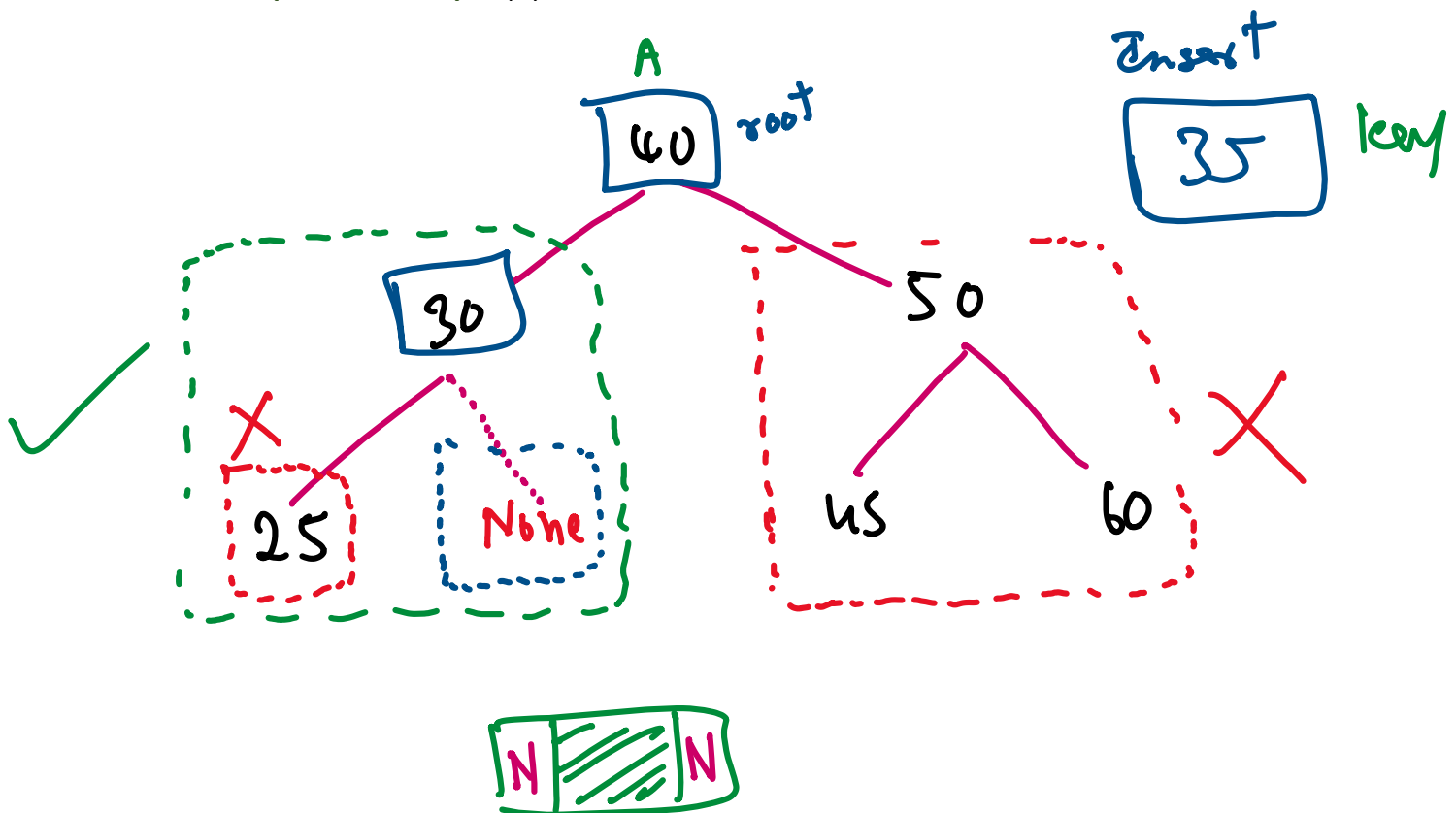
Saturday, December 20, 2025 12:29 PM

Steps:

- Start from **root**
- If **key < root.val** → go left
- If **key > root.val** → go right
- Insert at first **None** position

Time Complexity:

- **Average:** $O(\log n)$
- **Worst case (skewed tree):** $O(n)$



insert(30, 35)

A.left = insert(root.left, 35)

insert(30, 35)

35-N

30.right = insert(root.right, 35)

insert(None, 35)

35-N

12.5.2 - Searching

Saturday, December 20, 2025 12:29 PM

Steps:

- Compare key with **root.val**
- Go left or right accordingly
- Stop when found or reach **None**

Time Complexity:

- **Average:** $O(\log n)$
- **Worst case:** $O(n)$

12.5.3 - Deletion

Saturday, December 20, 2025 12:29 PM

Deletion is tricky because **BST property must be preserved**

Case 1: Leaf Node (No Children)

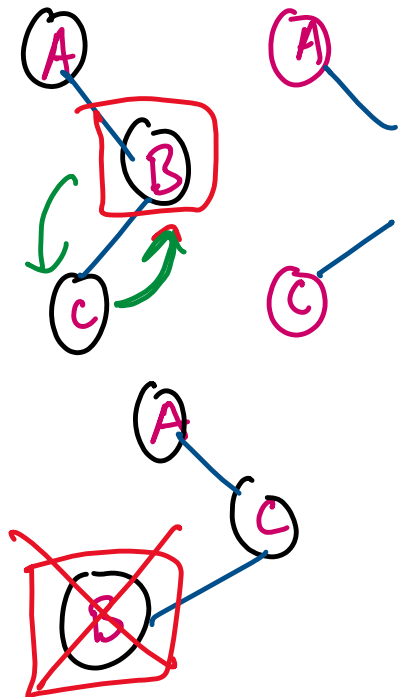
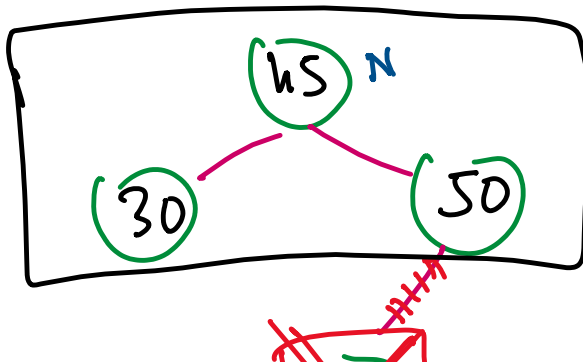
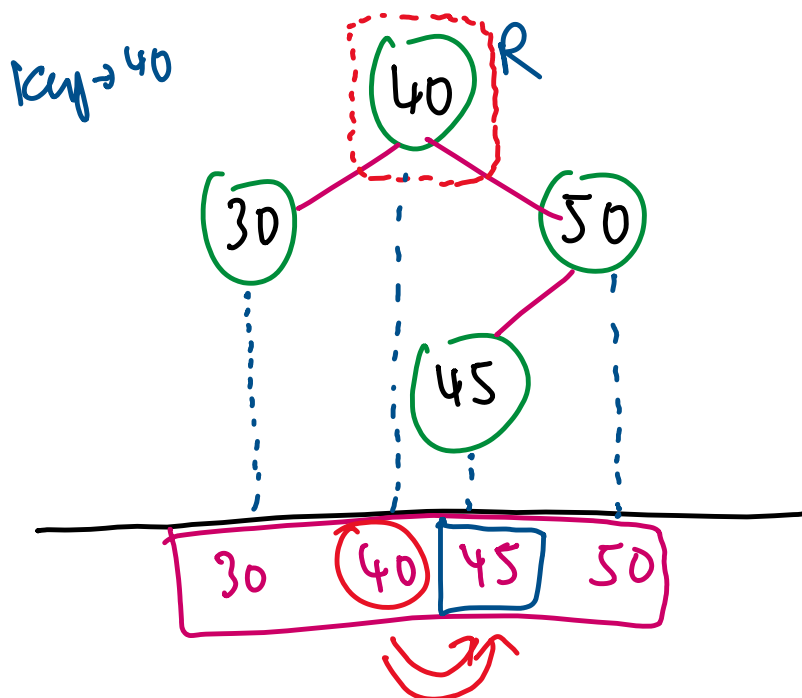
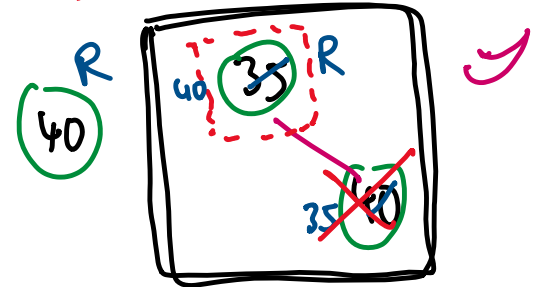
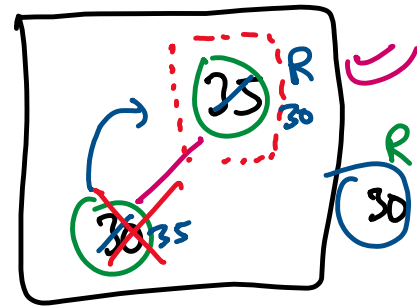
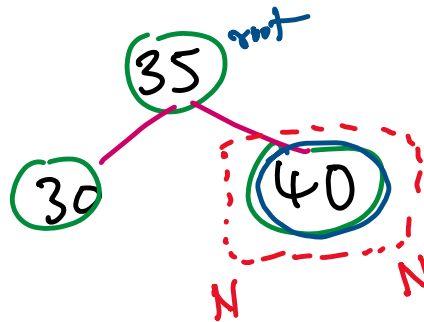
- Simply remove the node

Case 2: Node with One Child

- Replace node with its child

Case 3: Node with Two Children (Most Important)

- Find Inorder successor (smallest value in right subtree)
- Replace node's value with successor's value
- Delete successor node

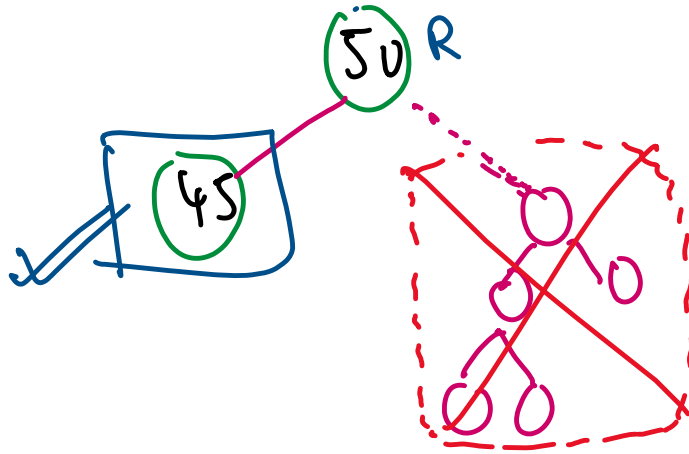


Smallest value greater than



Greater than
the node

1. Given the node / key
2. Visit / explore the right subtree
3. Visit the left-most node



12.6 - N-ary Tree

Saturday, December 20, 2025 11:59 PM

What is an N-ary Tree?

An **N-ary tree** is a tree where each node can have 0 to N children

- **Binary Tree:** max 2 children
- **N-ary Tree:** any number of children

N-ary Tree

↳ max no of children
any node can have

Traversal:

- **Preorder** (*Root* → *Children*)
- **Postorder** (*Children* → *Root*)
- **Level Order**

Node Structure:

```
class Node:
    def __init__(self, val):
        self.val = val
        self.children = []
```

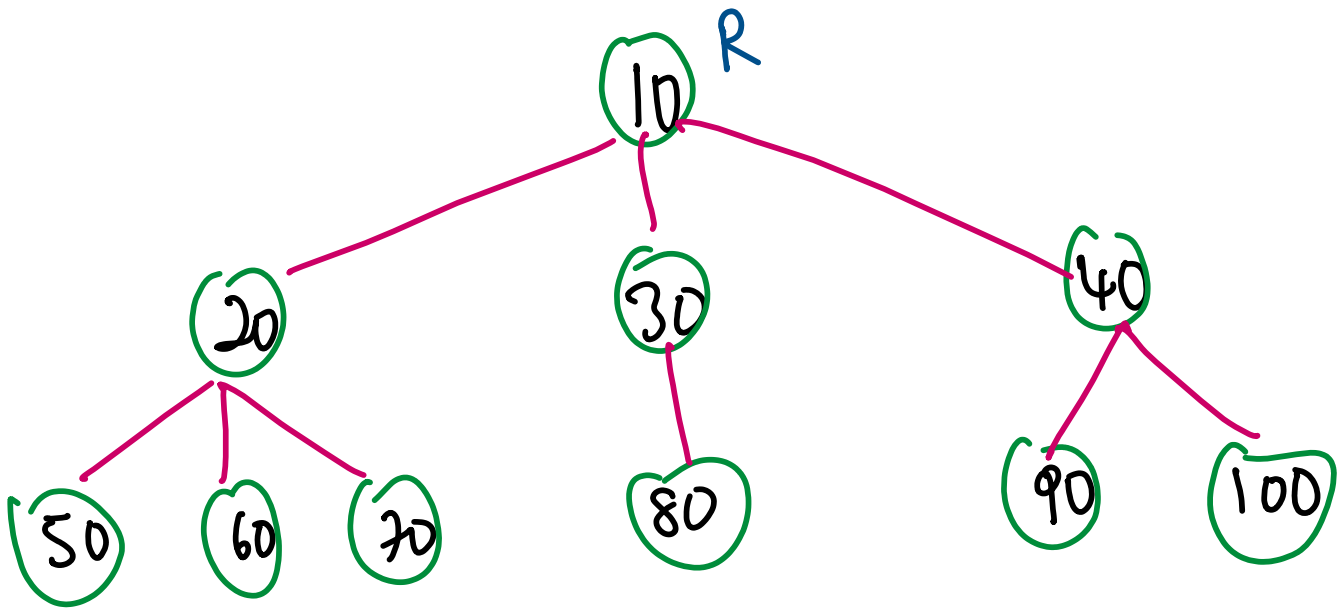
- **children** is a **list** ✓
- No fixed left/right
- Order of **children** may or may not matter

Key Applications:

- **File system** (folders → many subfolders)
- **Organization hierarchy**
- **XML / HTML DOM tree**
- **Trie** (special N-ary tree)

3-Ary Tree

m R



12.6.1 - Traversals

Sunday, December 21, 2025 12:10 AM

1. Preorder:

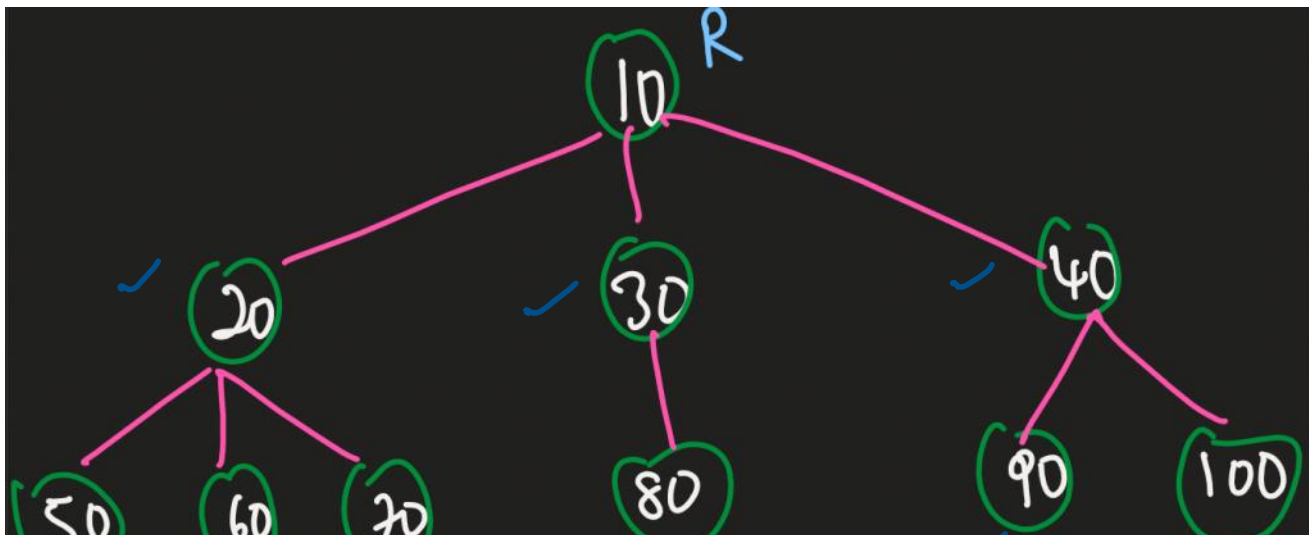
- Visit the root first
- Then, visit all children, one at a time
- **Use cases:**
 - Serialize tree
 - Copy tree
 - Prefix expressions

2. Postorder:

- Visit the children first
- Then, visit the root node
- **Use cases:**
 - Deleting tree
 - Aggregation problems (sum, height, DP)

3. Level Order:

- Natural *level by level* expansion
- Used in hierarchy / shortest path problems



~~80~~ ~~90~~ 100

10, 20, 30, 40, 50, 60, 70, 80, 90, 100

Level-Order Traversal

1. Check if Queue is empty
2. Dequeue
3. Process the dequeued node
4. Enqueue its child nodes

12.7 - Tree DP

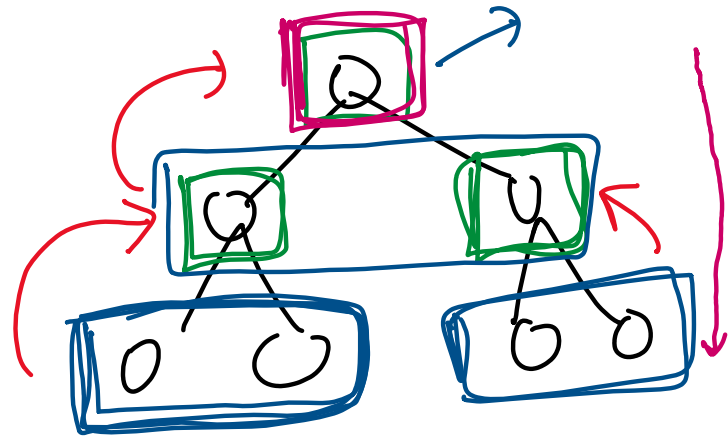
Sunday, December 21, 2025 12:14 AM

What is Tree DP?

- Each node computes an answer using its children's answers

Key Properties:

- No cycles → recursion can be used
- Each subtree is an independent problem
- Postorder traversal is the backbone of Tree DP



Why only Postorder for Tree DP?

- Tree DP needs the children's results before the parent can be computed
- Postorder is the only traversal that guarantees this

When to use Tree DP?

- Does the answer for a node depend on the children?
- Are there multiple choices at each node?
- Is there a need to return more than one value?

Tree DP Template:

```
def dfs(node):  
    # 1. Base case  
    if not node:  
        return BASE_VALUE  
  
    # 2. Compute results from children  
    child_results = []  
    for child in node.children:  
        child_results.append(dfs(child))  
  
    # 3. Combine child results  
    result = combine(child_results, node)  
    return result
```

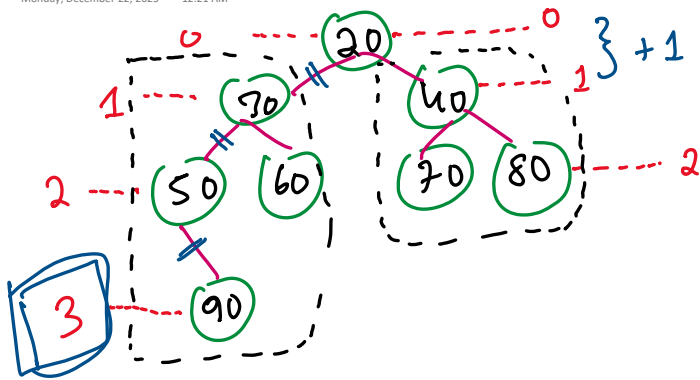
Base Condition

Process child nodes

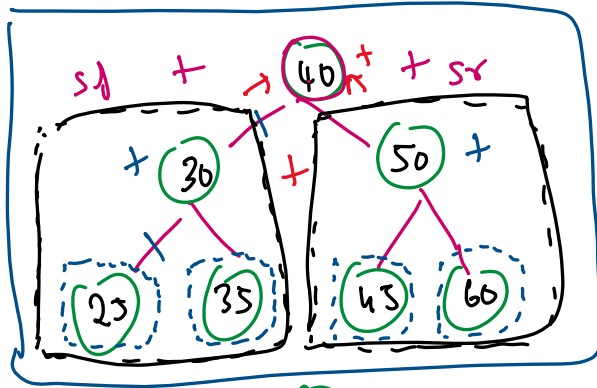
Combine results

1. Base Condition
2. Visit / Process child nodes
 - in child nodes

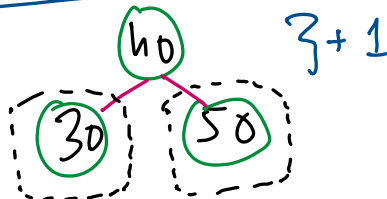
2. Visit/Process child room
3. Complete parent's child notes
4. Return result



1. Observe left & right subtrees
2. At root, depth is zero
3. Obtain $\max(\text{left}, \text{right}) + 1$



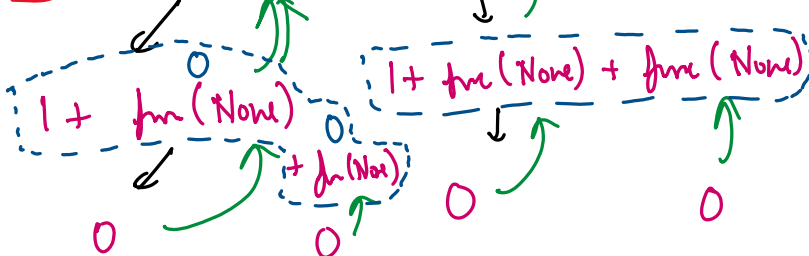
$\text{sum-left} + \text{sum-right} + \text{node.value}$



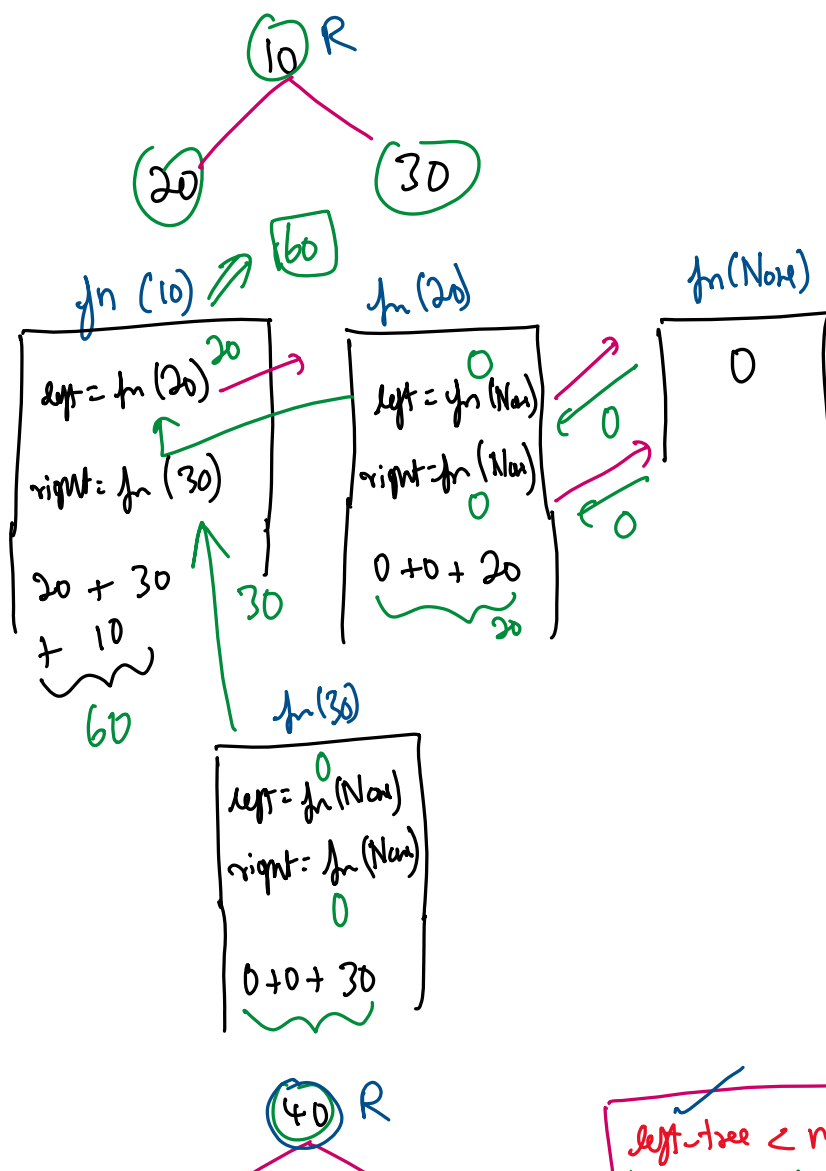
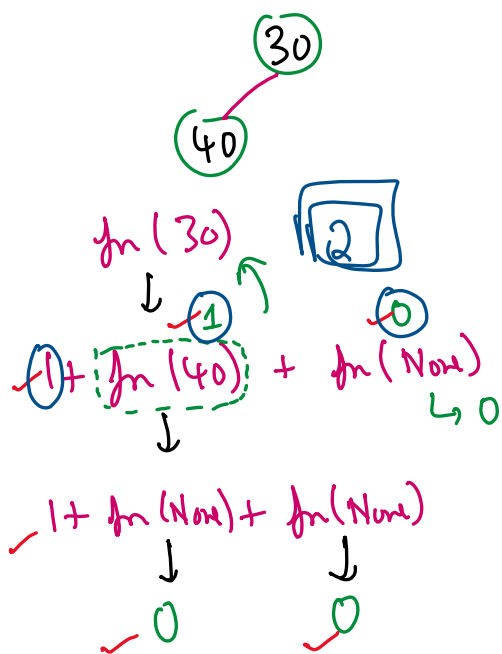
$\text{return } 1 + \text{count}(\text{left}) + \text{count}(\text{right})$

$\text{func}(40)$

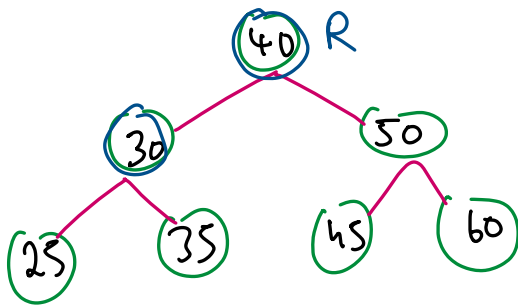
$$1 + \text{func}(30) + \text{func}(50) = 3$$



(20)



left tree < node.val < right tree



$\text{left-tree} < \text{node.val} < \text{right-tree}$

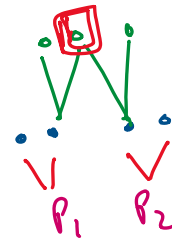
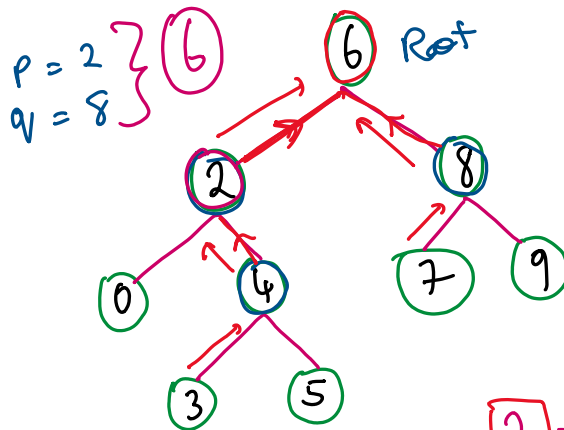
node.val ✓
left (low)
right (high)

node → node.left ✓
low → low ✓
high → node.value ✓

left

node → node.right ✓
low → node.value ✓
high → high ✓

right



$p=2$
 $q=4$ } (2)

2 → 6
4 → 2 → 6

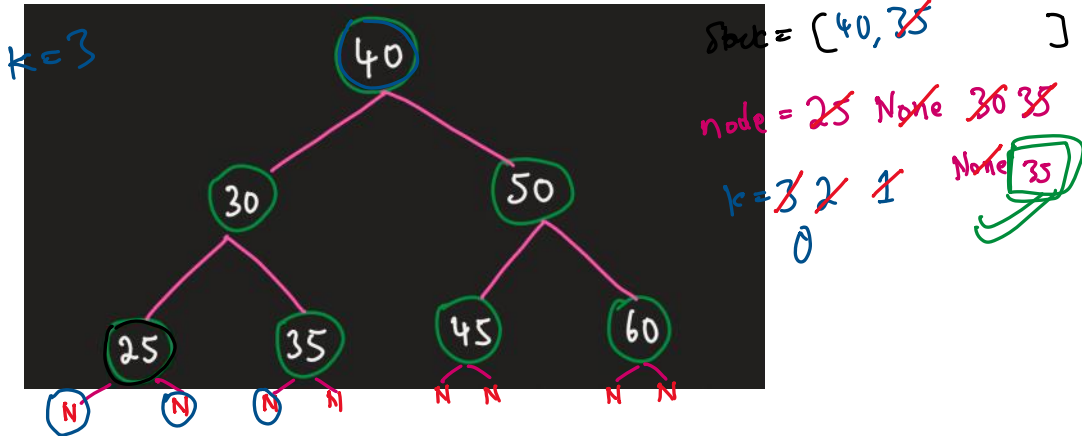
i) $p < \text{node} \ \& \ q < \text{node}$:
left sub tree

ii) $p > \text{node} \ \& \ q > \text{node}$:
right sub tree

right sub tree

return node

3 → 4 → 2 → 6
2 → 8 → 6

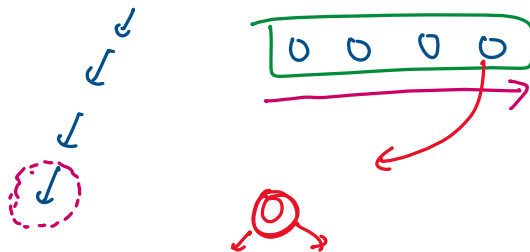
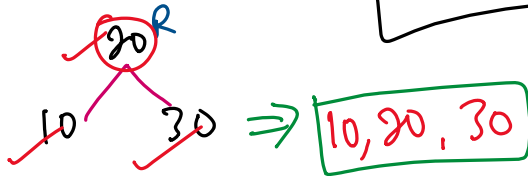


10 17 23 25 34

BST → Inorder Traversal → Ascending Order

Recursive X
Iterative ✓

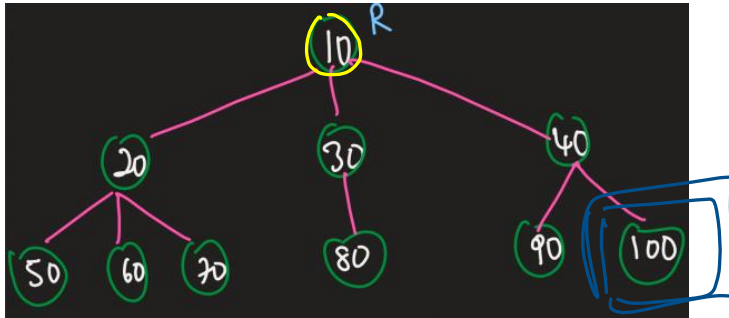
Inorder Traversal
Left, Root, Right



Stack!!

Stack!!

$$k = 3 \times 0$$



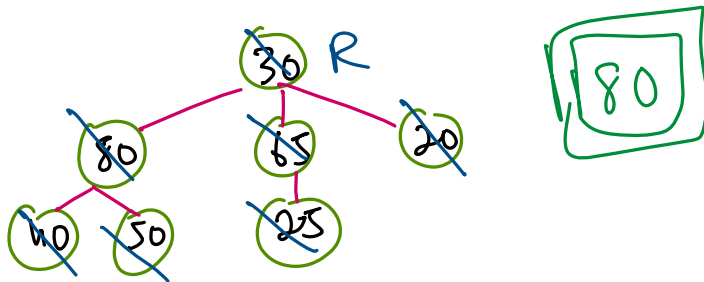
Base Condition $\rightarrow -inf$

max_val $\rightarrow$ node.val

for child in node.children:

max_val = max(max_val, fn(child))

return max_val



fn(30)

res = ~~30~~ 80

[80, 65, 20]

max(30, fn(80))

80

max(80, fn(65))

fn(80)

res = 80

[40, 50]

res = max(80, fn(40))

40

res = max(80, fn(50))

50

fn(40)

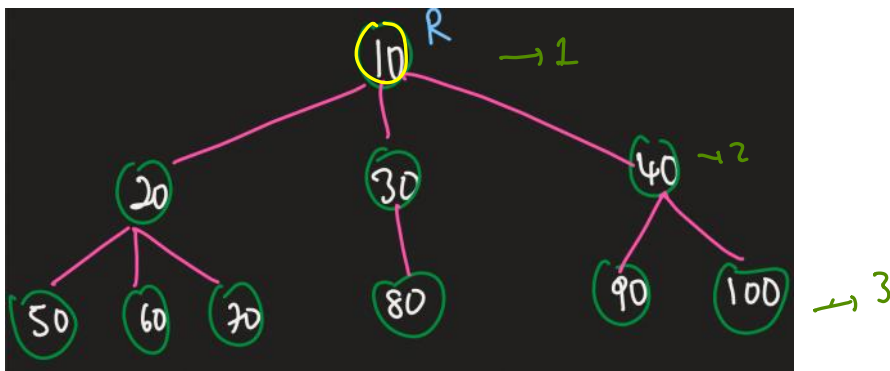
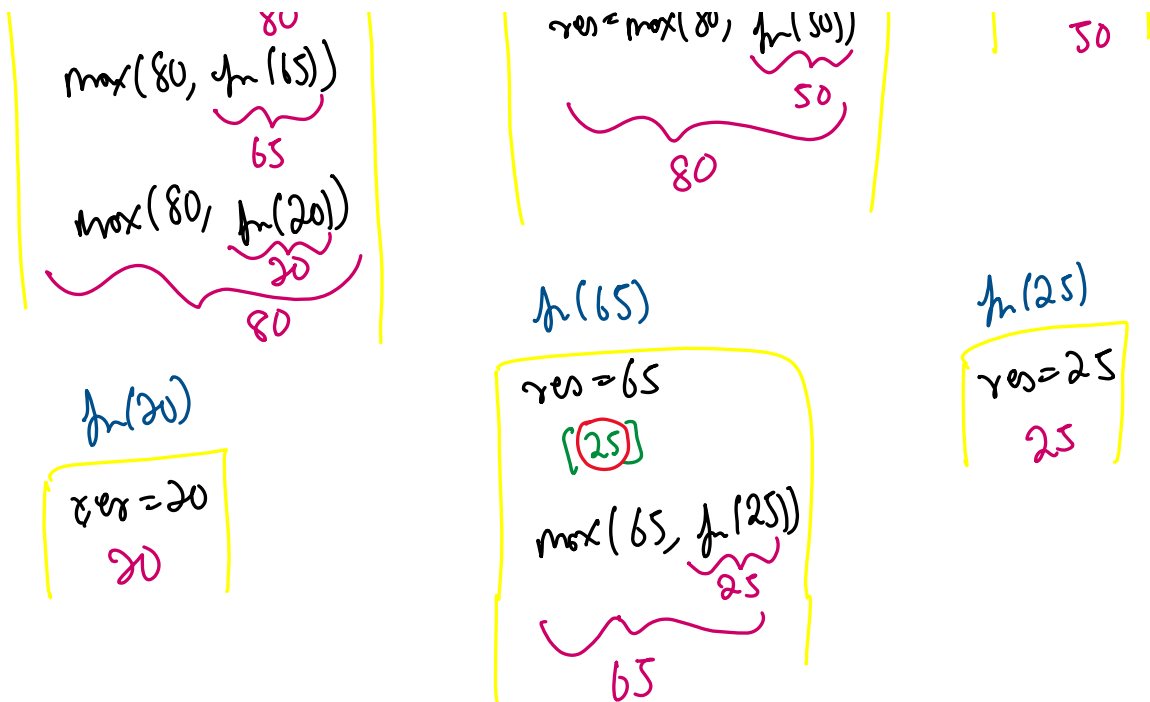
res = 40

40

fn(50)

res = 50

50

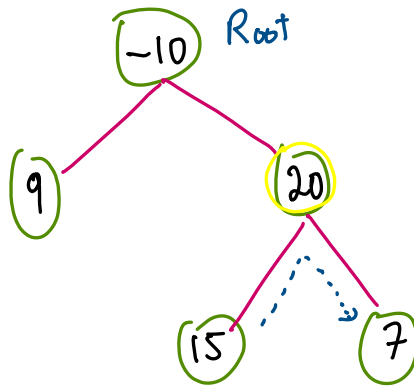


Base Condition $\rightarrow 0$

heights = []
 for each child:
 heights.append(h(child))
 return 1 + max(heights)

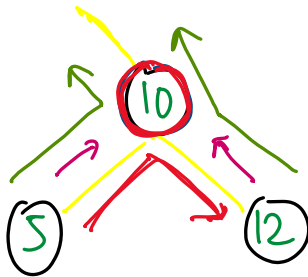
(-10) Root

$$15 + 20 + (-10) = 25$$



$$\begin{aligned}
 15+20+(-10) &= 25 \\
 7+20+(-10) &= 17 \\
 15+20 &= 35 \\
 7+20 &= 27 \\
 \boxed{15+7+20} &= \boxed{42}
 \end{aligned}$$

Paths $\rightarrow$ Can start & end at any node



- i) Path ends at node
- ii) Path goes through node upwards

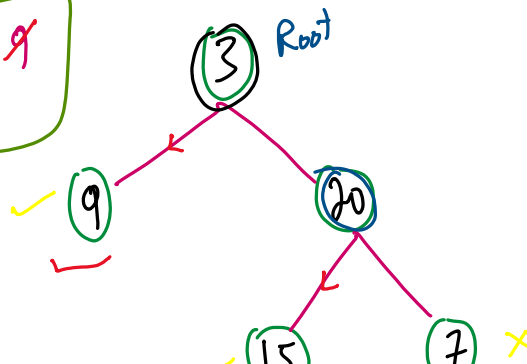
$$\begin{aligned}
 5 &\rightarrow 10 \\
 12 &\rightarrow 10
 \end{aligned}$$

$fn(node) \rightarrow$ res of tree/node
 $fn(node.left) \rightarrow$ res of left subtree
 $fn(node.right) \rightarrow$ res of right subtree
 recursive action

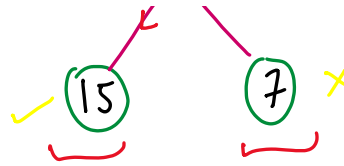
res $\rightarrow$ node.value +
left gain +
right gain

res $\rightarrow$ node.value +
 $\max(\text{left-gain}, \text{right gain})$

Sum = 9
24



$9+15 \rightarrow 24$



1. Node should be leaf node
2. Node must be left child

$f(3)$

$$\begin{aligned}
 \text{total} &= \cancel{0} + 24 \\
 \text{total} &+= f(9) \quad \left. \begin{array}{l} 0 \\ 15 \end{array} \right\} \\
 \text{total} &+= f(20) \quad \left. \begin{array}{l} 0 \\ 15 \end{array} \right\} \\
 &\underline{\hspace{1cm}} \\
 &\quad \boxed{24}
 \end{aligned}$$

$f(9)$

$$\begin{aligned}
 \text{total} &= 0 \\
 \text{total} &+= f(\text{None}) \quad \left. \begin{array}{l} 0 \\ 30 \end{array} \right\} \\
 \text{total} &+= f(\text{None}) \quad \left. \begin{array}{l} 0 \\ 30 \end{array} \right\} \\
 &\underline{\hspace{1cm}} \\
 &\quad 0
 \end{aligned}$$

$f(\text{None})$

$$\begin{aligned}
 &\boxed{0}
 \end{aligned}$$

$f(20)$

$$\begin{aligned}
 \text{total} &= \cancel{0} + 15 \\
 \text{total} &+= f(15) \quad \left. \begin{array}{l} 0 \\ 30 \end{array} \right\} \\
 \text{total} &+= f(7) \quad \left. \begin{array}{l} 0 \\ 30 \end{array} \right\} \\
 &\underline{\hspace{1cm}} \\
 &\quad 15
 \end{aligned}$$

$f(15)$

$$\begin{aligned}
 \text{total} &= 0 \\
 \text{total} &+= f(\text{None}) \quad \left. \begin{array}{l} 0 \\ 30 \end{array} \right\} \\
 &\underline{\hspace{1cm}} \\
 &\quad 0
 \end{aligned}$$

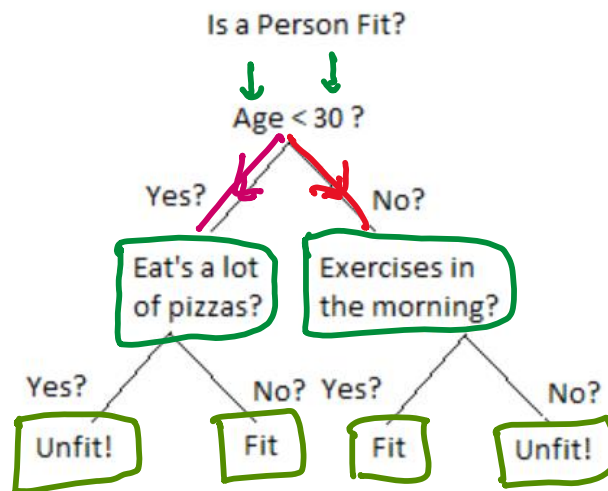
$f(7)$

$$\begin{aligned}
 \text{total} &= 0 \\
 \text{total} &+= f(\text{None}) \quad \left. \begin{array}{l} 0 \\ 30 \end{array} \right\} \\
 \text{total} &+= f(\text{None}) \quad \left. \begin{array}{l} 0 \\ 30 \end{array} \right\} \\
 &\underline{\hspace{1cm}} \\
 &\quad 0
 \end{aligned}$$

Applications of Tree Data Structures in Data Science & Machine Learning:

- Decision Trees:

- These can be binary or multi-branch trees depending on problem statement
- Each node results from a feature-value split
- Each edge corresponds to an outcome/decision
- Leaf nodes state the final prediction value



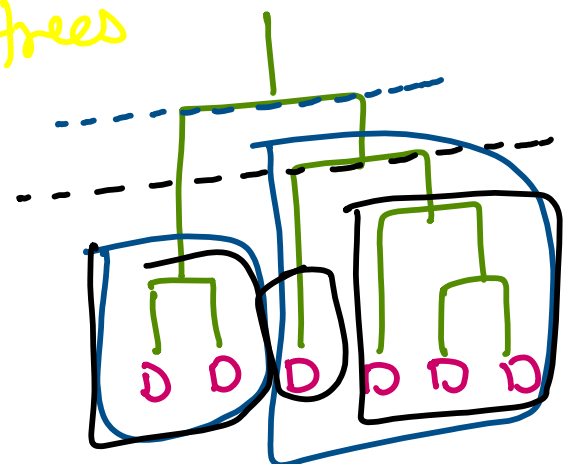
- Random Forests

- Gradient Boosted Trees

} collection of multiple
decision trees

- Hierarchical Clustering:

- Forms a tree-like structure called a Dendrogram
- Clusters of elements forms the nodes
- Cutting the tree at a certain heights gives different clusters

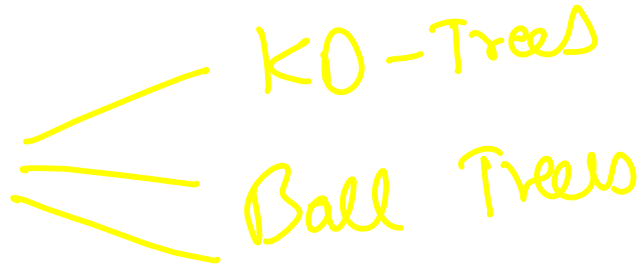


- NLP Parse Trees & Syntax Trees:

- Grammar-based NLP
- Semantic analysis
- Syntax-aware transformers

- Data Indexing & Search in ML Systems:

- Nearest Neighbor Search
- Recommendation systems
- Similarity search



13 - Graphs

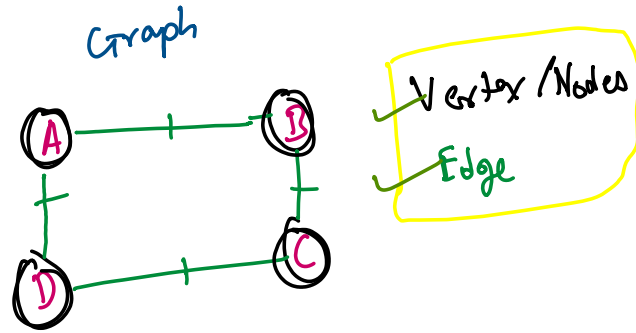
Sunday, December 28, 2025

12:21 AM

- Introduction
- Types of Graphs
- Degree of Graph
- Graph Connectivity
- Graph Representation
- Graph Traversal
- Traversal Variants & Optimizations
 - *Multi-source BFS*
 - *0-1 BFS*
 - *BFS vs DFS decision making*
- Shortest Path Algorithms
 - *Dijkstra's Algorithm*
 - *Bellman–Ford Algorithm*
 - *Floyd–Warshall*
- Advanced Graph Concepts
- State-Space Graphs
- Directed Acyclic Graph (DAG) Algorithms
- Coding Problems

What is a Graph?

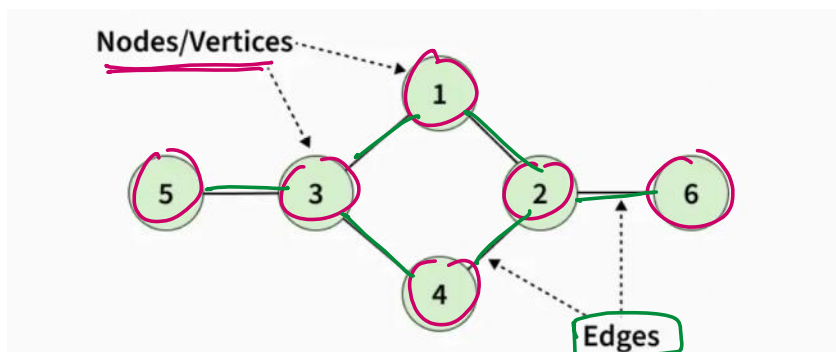
- A graph is a non-linear data structure used to model relationships between entities
- Unlike arrays, linked lists, or trees, graphs do not impose a strict hierarchy or order
- Mathematically, a graph is defined as $\rightarrow G = (V, E)$
 - **V (Vertices / Nodes)** $\rightarrow$ A finite set of points
 - **E (Edges)** $\rightarrow$ A set of connections between pairs of vertices
- Example:
 - $V = \{A, B, C, D\}$
 - $E = \{(A, B), (B, C), (C, D), (D, A)\}$

Vertices (Nodes):

- Vertices are the fundamental units of the graph
- Vertices are also known as vertex or nodes
- Vertices represent **entities** in a system
- Can store **values or labels**
- Can have **attributes** (weight, color, metadata)
- Total number of vertices = $|V|$

Edges (Connections):

- Edges represent **relationships between vertices**
- Edges are used to connect two nodes of the graph
- Edges can connect any two nodes in any possible way, there are no rules
- Every edge can be labelled/unlabelled
- Edges are also known as **arcs**

Applications:

- **Social Networks:** Represent users and their connections; used to find mutual friends, suggest new connections, and detect communities
- **Computer Networks:** Model routers and data links; used for efficient routing, fault detection, and network optimization
- **Transportation Networks:** Represent cities and routes; used to find shortest or fastest paths and plan optimal travel routes
- **Neural Networks:** Represent neurons and synapses; used to simulate learning, brain behavior, and data processing
- **Compilers:** Represent data dependencies and control flows; used for optimization, register allocation, and code analysis
- **Robot Path Planning:** Represent states and transitions; used to compute the safest or shortest route for autonomous movement
- **Project Dependencies:** Represent tasks and dependencies; used in topological sorting to determine the correct execution order

- **Network Optimization:** Represent network nodes and links; used to minimize cost, reduce latency, and improve efficiency

Advantages of Graphs:

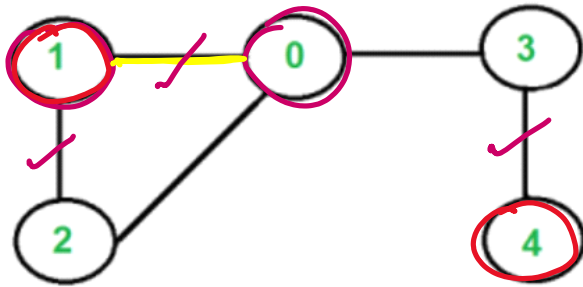
- **Flexibility:** Unlike arrays, linked lists, or trees, graphs have no restrictions and can represent any type of relationship
- **Model real-world problems:** Useful for pathfinding, data clustering, network analysis, and machine learning
- **Represent items and relationships:** Any set of items and their connections can be modeled as a graph
- **Simplifies complex data:** Graphs make complex relationships easy to visualize and understand

13.2 - Types of Graphs

Sunday, December 28, 2025 10:05 AM

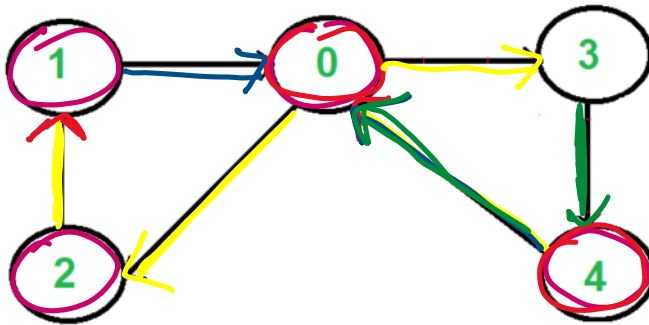
1. Undirected Graph:

- A graph in which edges do not have any direction
- Nodes are unordered pairs in the definition of every edge
- Edges in an undirected graph are *bidirectional* in nature
- There is no concept of a "parent" or "child" vertex as there is no direction to the edges
- An undirected graph may contain loops, which are edges that connect a vertex to itself



2. Directed Graph:

- A graph in which edges have direction
- One-way relationship between vertices
- A directed graph can contain cycles
- Paths in a directed graph follow the direction of the edges

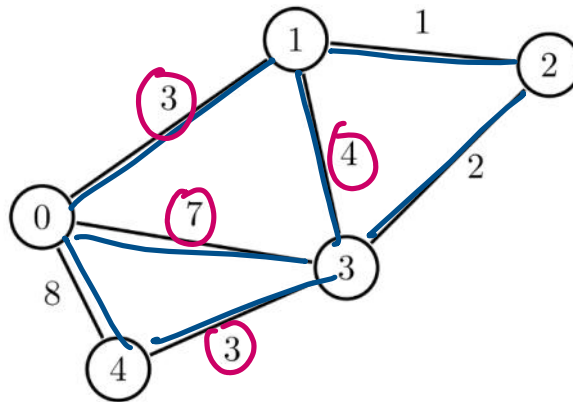


0

In-degree = 2
Out-degree = 2

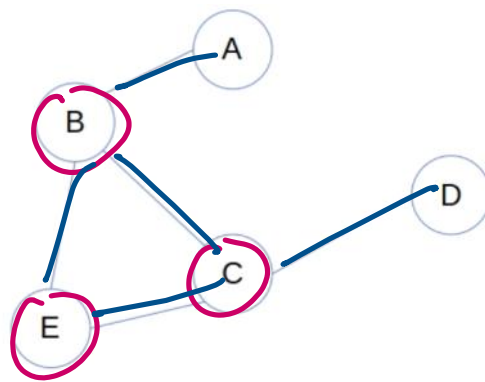
3. Weighted Graph:

- A weighted graph is one where the edges are assigned some weights
- Weights can represent cost, distance, or any other relative measuring unit



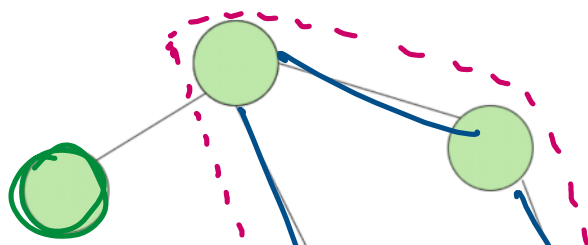
4. Unweighted Graph:

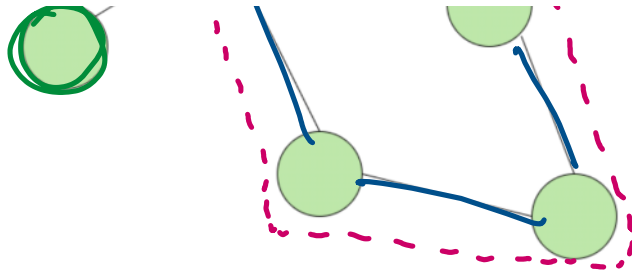
- Graph in which the edges do not have weights or costs associated with them
- Simply represent the presence of a connection between two vertices



5. Cyclic Graph:

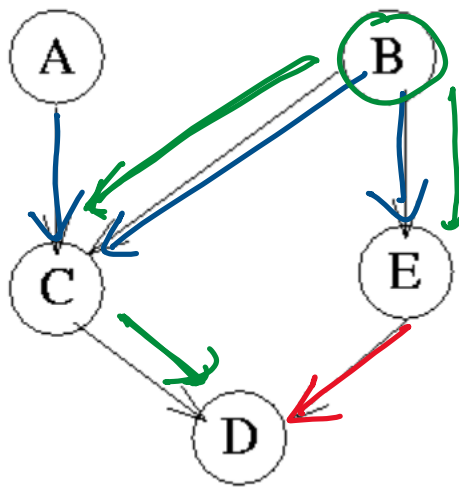
- A cyclic graph contains one or more cycles or closed paths
- A cyclic graph can be either directed or undirected
 - **Directed Cyclic Graph:** edges have a direction, and the cycle must follow the direction of the edges
 - **Undirected Cyclic Graph:** edges have no direction, and the cycle can go in any direction
- A cyclic graph may have multiple cycles of different lengths and shapes; some cycles may be contained within other cycles





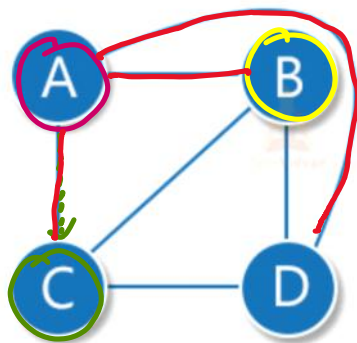
6. Acyclic Graph:

- An acyclic graph is a graph that contains no cycles or closed loops



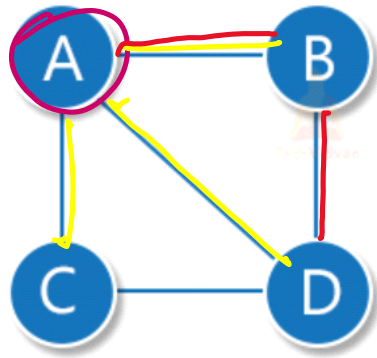
7. Complete Graph:

- A complete graph is a graph in which every node is adjacent to every other node
- A complete graph has $\frac{n(n-1)}{2}$ edges



8. Incomplete Graph:

- An incomplete graph is a graph in which every node is not adjacent to every other node



13.3 - Degree

Sunday, December 28, 2025

10:14 AM

- In a graph, degree is defined with respect to a vertex
- It is the number of edges incident (connected) to that vertex

Degree in an Undirected Graph:

- Number of edges connected to a vertex

Degree in a Directed Graph:

- In directed graphs, degree is split into two types:
 - **In-degree:** Number of edges *coming into* a vertex
 - **Out-degree:** Number of edges *going out of* a vertex

Degree Sequence:

{ }

- A degree sequence is a **list of degrees of all vertices** in a graph
- Usually written in descending order (largest to smallest)

13.4 - Connectivity

Sunday, December 28, 2025 10:39 AM

What is Graph Connectivity?

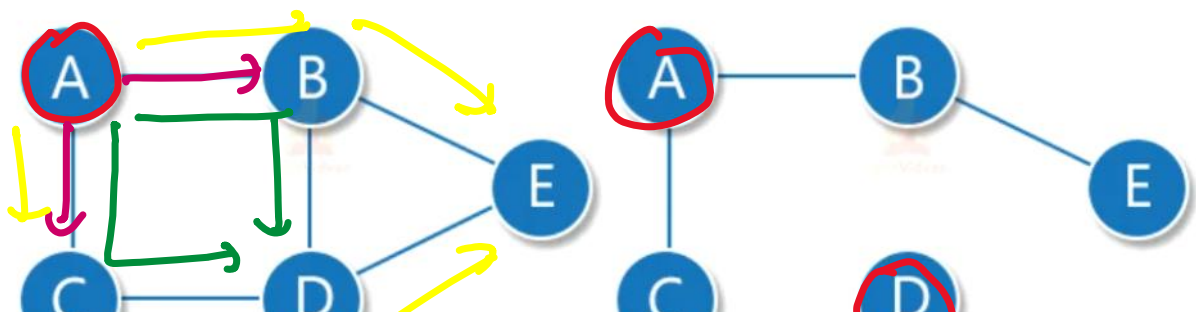
- Connectivity dictates **how vertices are linked together** in a graph
- There are two types of graphs based on this:
 - *Connected Graph*
 - *Disconnected Graph*

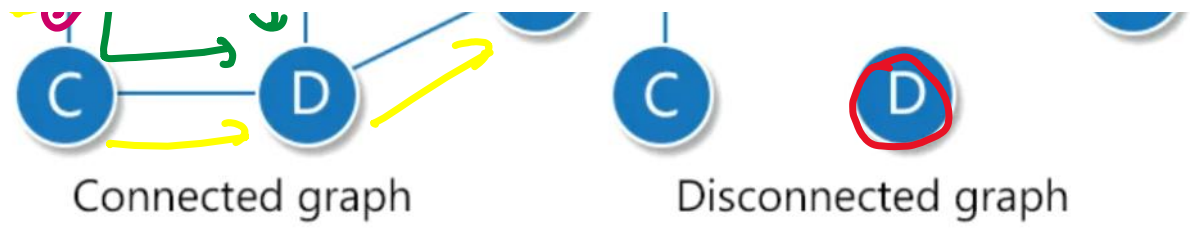
1. Connected Graph:

- A graph is connected, if every vertex is reachable from every other vertex
- There is at least one path between any two vertices in the graph
- The path between vertices can be direct or indirect

2. Disconnected Graph:

- A graph is disconnected if at least one pair of vertices cannot reach each other
- This occurs when the graph is split into separate parts with no edges between them





13.5 - Representation

Sunday, December 28, 2025 10:48 AM

What is Graph Representation?

- A graph representation is how we store a graph in computer memory
- Representation enables implementation of algorithms like BFS, DFS, Dijkstra, etc.

Why Do We Need Graph Representation?

- Different representations help us:
 - *Save memory*
 - *Access neighbors quickly*
 - *Run graph algorithms efficiently*
- There is no single best representation—it depends on:
 - Number of vertices (V)
 - Number of edges (E)
 - Type of graph (dense or sparse)

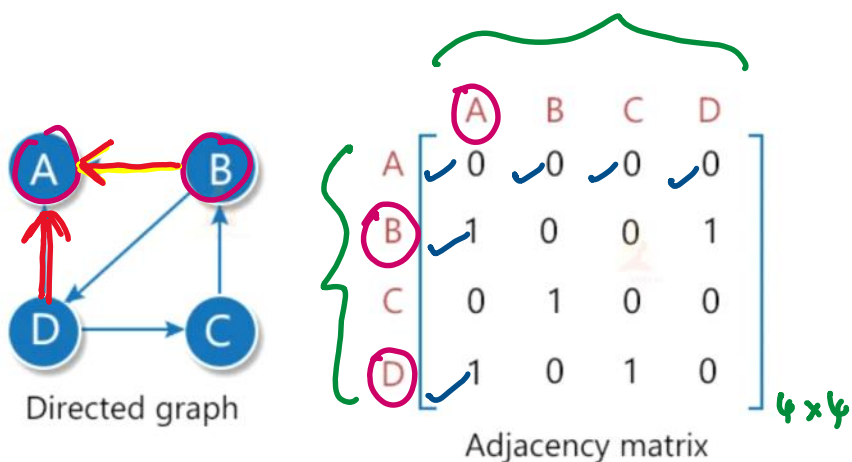
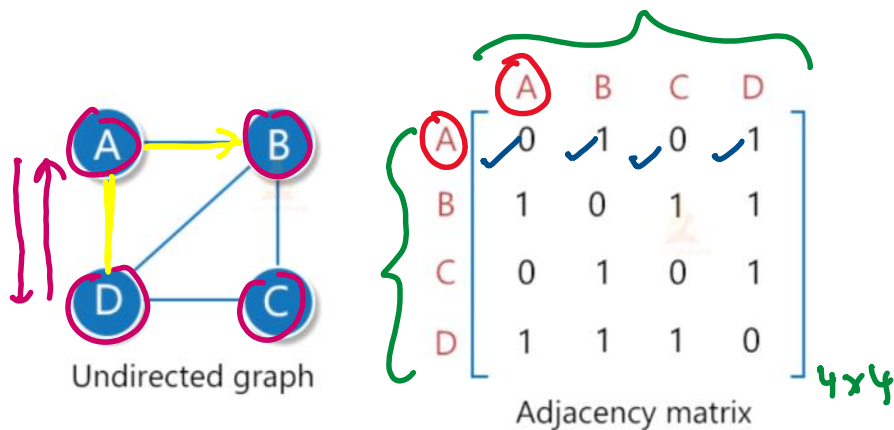
Most Common Graph Representations:

- *Adjacency Matrix*
- *Adjacency List*

13.5.1 - Adjacency Matrix

Sunday, December 28, 2025 10:52 AM

- An adjacency matrix is a way of representing a graph as a boolean matrix of (0's and 1's)
- The matrix represents the mapping between various edges and vertices
- The order of the matrix is given as $n \times n$ where n is the number of nodes in the graph
- In the matrix, each row and column represents a vertex and the values determine the presence of edges
- The representation varies for directed and undirected graphs, despite similar structure



Advantages of Adjacency Matrix:

- Performing operations on a matrix are easier as compared to performing them on the list or other data structure
 - Adding and removing edges
 - Checking if the edges are present in the graph
- Can perform heavy matrix operations easily on modern GPUs

- Efficient even if the graph is dense and the number of edges present in the graph is large

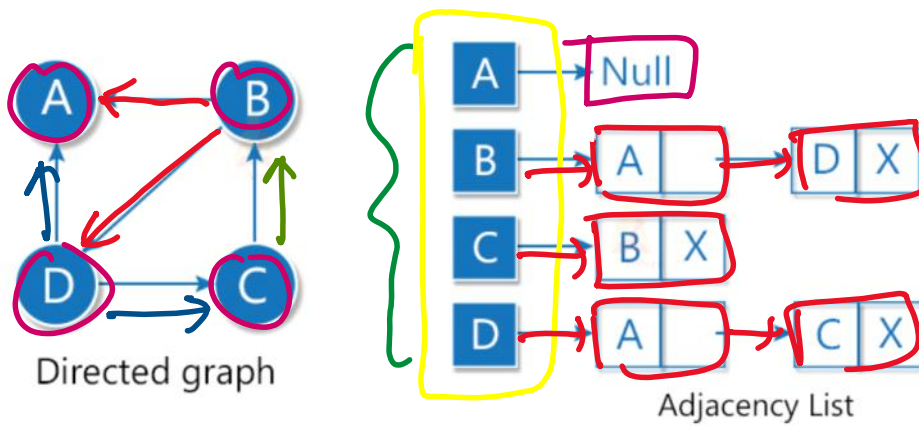
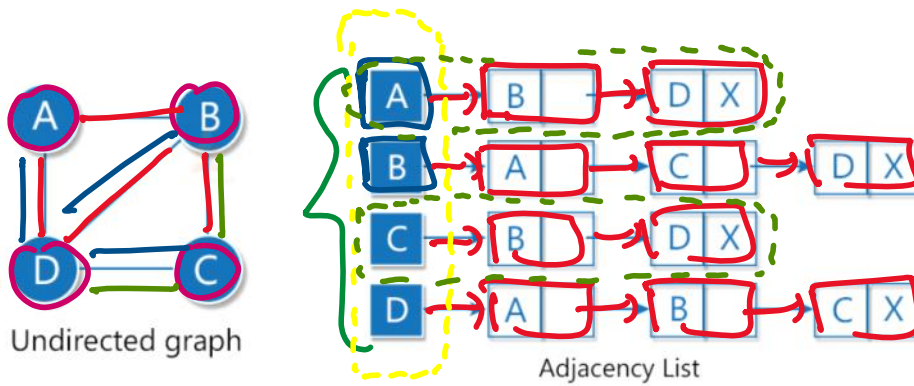
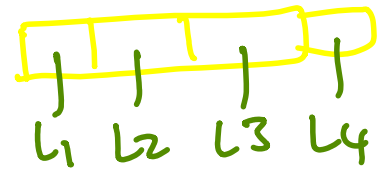
Disadvantages of Adjacency Matrix:

- In the adjacency matrix method, certain operations are computationally heavy and time consuming to perform
- Matrix representation requires a large amount of additional memory even if the number of edges in a graph is less

13.5.2 - Adjacency List

Sunday, December 28, 2025 10:52 AM

- The adjacency list is an array of linked lists
- {The array denotes the total vertices/nodes in the graph}
- Each linked list denotes the vertices connected to a particular node



13.6 - Traversal

Tuesday, December 30, 2025 12:57 AM

What is Graph Traversal?

Graph Traversal is the process of visiting all vertices (nodes) of a graph systematically

Applications of Graph Traversal:

- ✓ *• Searching paths*
- ✓ *• Detecting cycles*
- ✓ *• Finding connected components*
- ✓ *• Shortest path algorithms*
- ✓ *• Topological sorting*
- ✓ *• Real-world problems (maps, networks, recommendations)*

Fundamental Graph Traversal Techniques:

1. Depth First Search (DFS)
2. Breadth First Search (BFS)

TECHNIQUE	DATA STRUCTURE	NATURE	USAGE
DFS	Stack / Recursion	Deep first	Cycle detection, Topological sort
BFS	Queue	Level by level	Shortest path (unweighted graphs)

13.6.1 - DFS

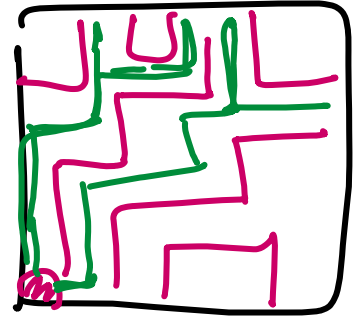
Tuesday, December 30, 2025 1:01 AM

What is DFS?

- Graph traversal algorithm that explores as deep as possible along one path before backtracking
- Starts from a source vertex, visits a neighbor, then a neighbor's neighbor, and so on, until no unvisited neighbors remain; then it backtracks
- DFS works best with *Adjacency List*

Intuition:

- Exploring a maze
- Going deep into one corridor before trying others



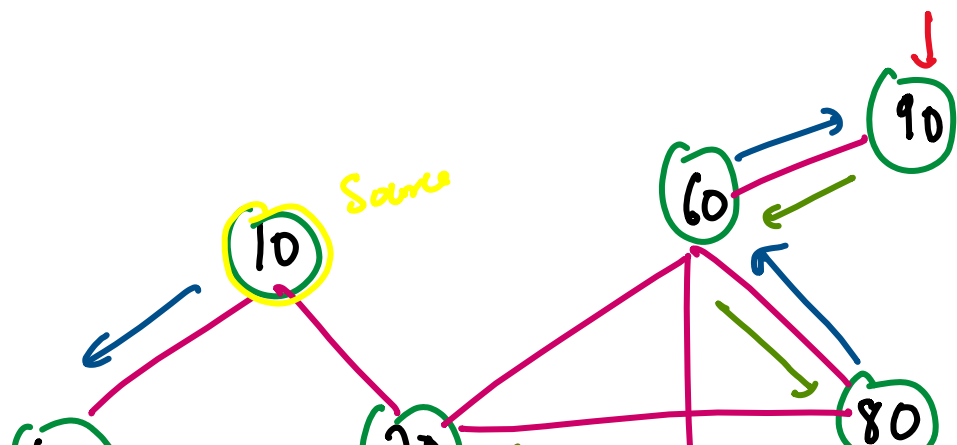
Algorithm:

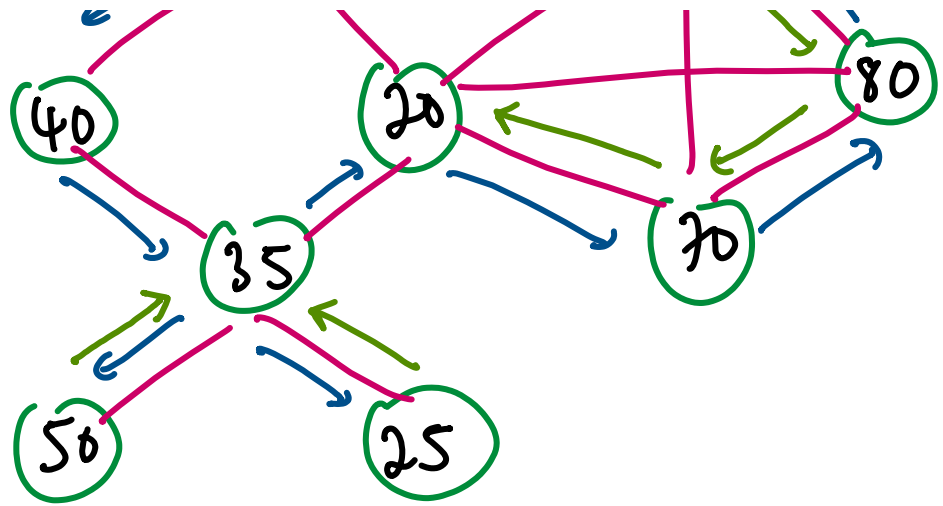
1. Create a visited set/array
2. Start DFS from a source node
3. Mark the current node as visited
4. For each adjacent node:
 - If not visited → perform DFS on it

Time and Space Complexity:

- Time: $O(V + E)$ ✓
- Space: $O(V)$ ✓

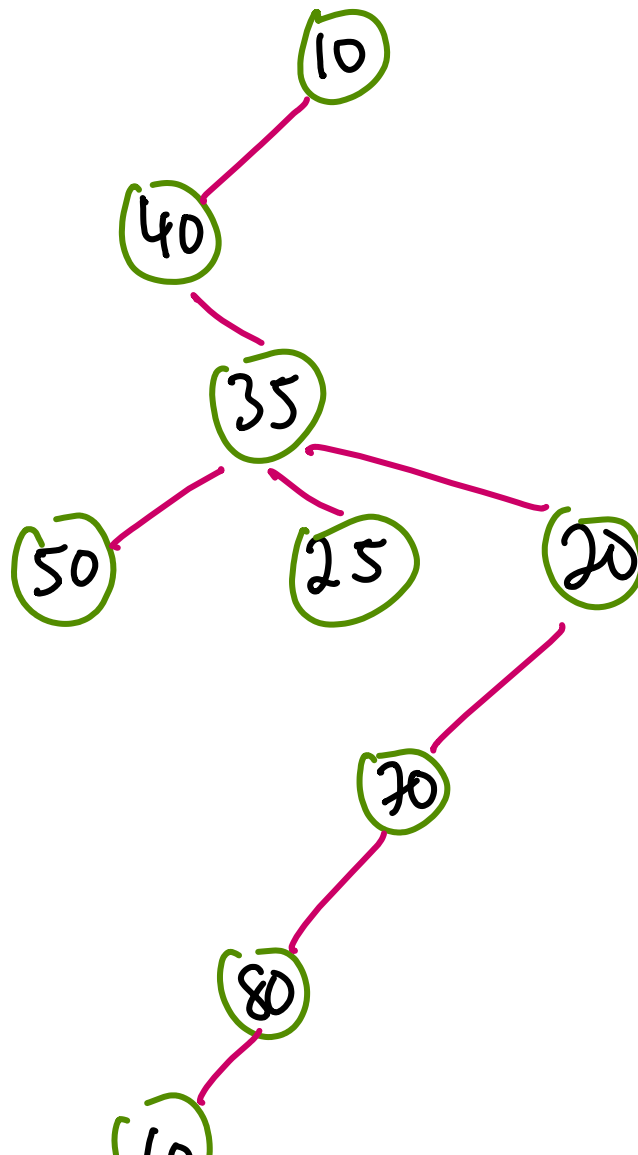
$V \rightarrow$ vertex
 $E \rightarrow$ edge

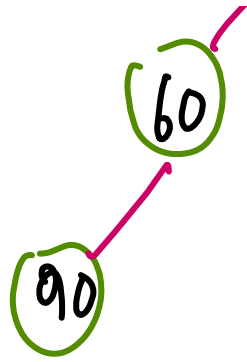




Path:

10, 40, 35, 50, 25, 20, 70, 80, 60, 90





13.6.2 - BFS

Tuesday, December 30, 2025 1:01 AM

What is BFS?

- Graph traversal algorithm that explores all neighbors of a node first, then their neighbors
- Traversal happens level by level starting from a source node
- Involves a Queue (FIFO), which ensures level-order traversal
- Ideal for shortest path problems in unweighted graphs

Algorithm:

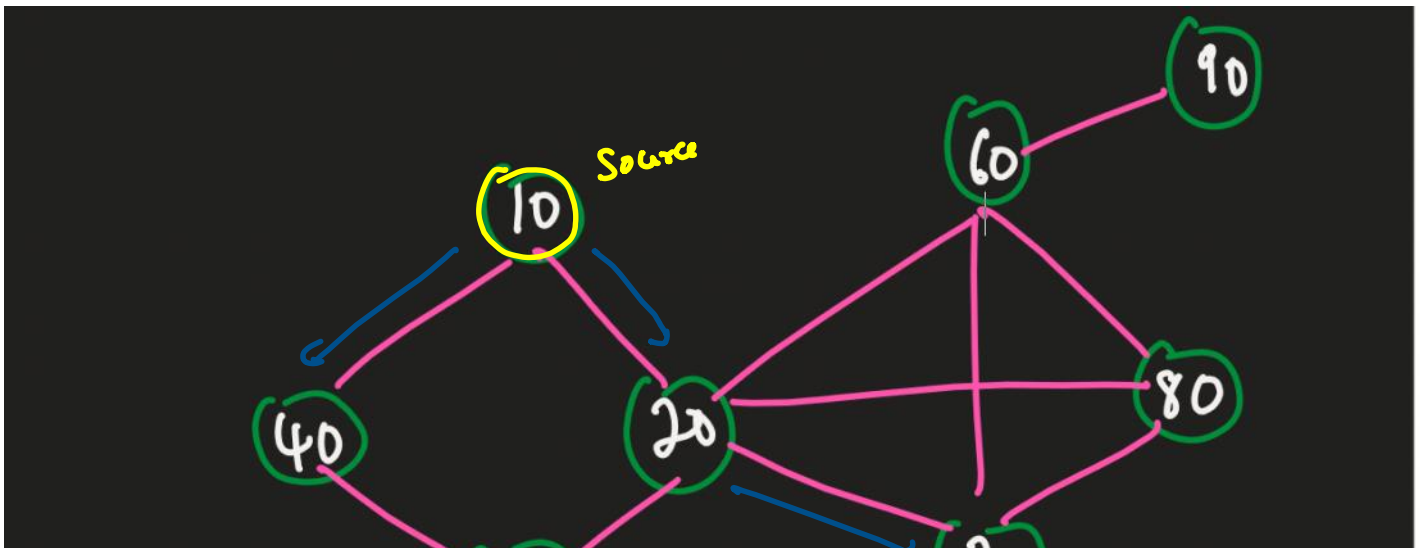
1. Start from a source node
2. Mark it as visited
3. Push it into a queue
4. While the queue is not empty:
 - Dequeue a node
 - Visit all its unvisited neighbors
 - Mark them visited and enqueue them

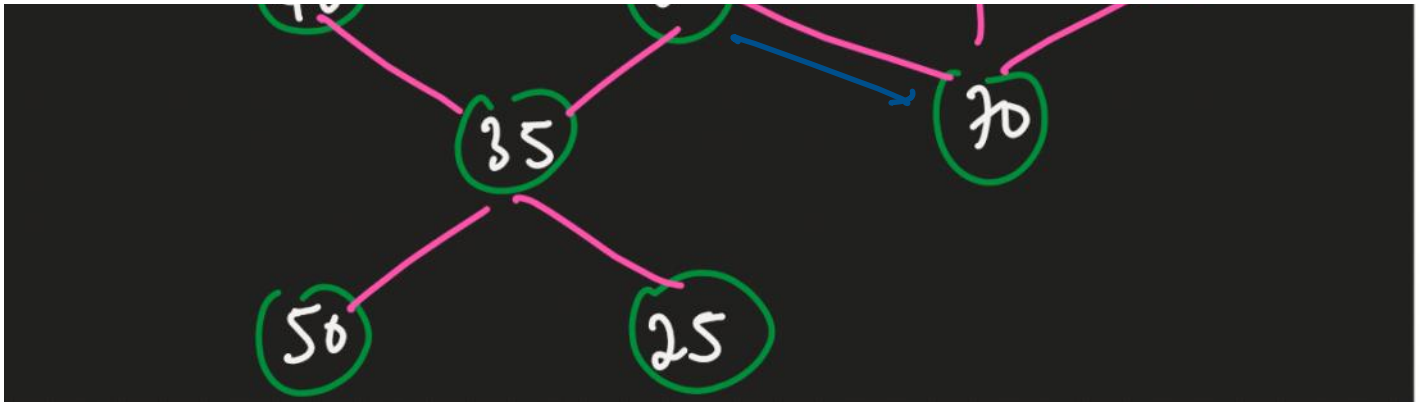
} Initial step

} Iterative steps

Time and Space Complexity:

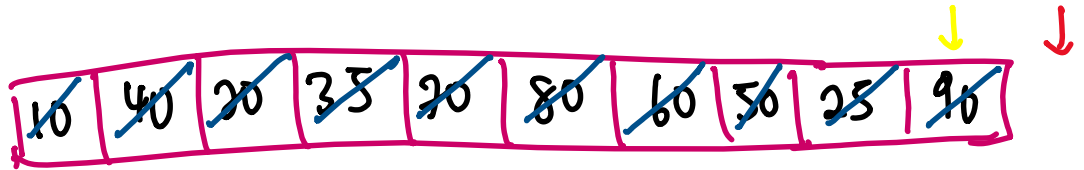
- Time: $O(V + E)$
- Space: $O(V)$





Queue:

front
rear



Path: 10, 40, 20, 35, 70, 80, 60, 50, 25, 90

Ans: 10 40 20 35 70 80 60 50 25 90

13.6.3 - Multi-source BFS

Wednesday, December 31, 2025 12:16 AM

What is Multi-Source BFS?

In normal BFS, we start from one source node whereas in Multi-Source BFS, we start BFS from multiple sources at the same time

When to use Multi-Source BFS?

- There are multiple starting points
- We want the minimum distance to the nearest source

Algorithm:

1. Identify all source nodes
2. Add all sources to queue
3. Mark all sources visited
4. Run normal BFS

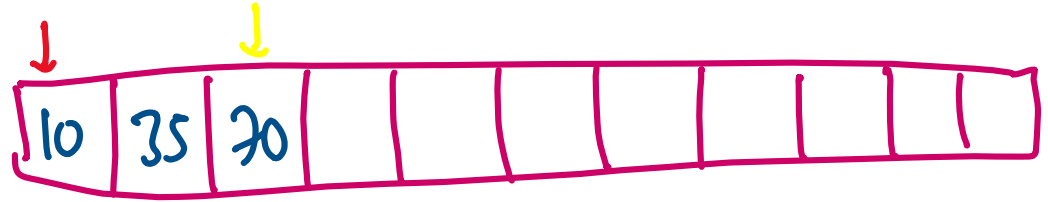
Time and Space Complexity:

- Time: $O(R \times C)$
- Space: $O(R \times C)$





Queue :



start
front

{ n1:
n2:
n3:
:
}

Min. distance b/w node &
its nearest source

$a \leftarrow^{+1} n1$

13.6.4 - 0-1 BFS

Wednesday, December 31, 2025

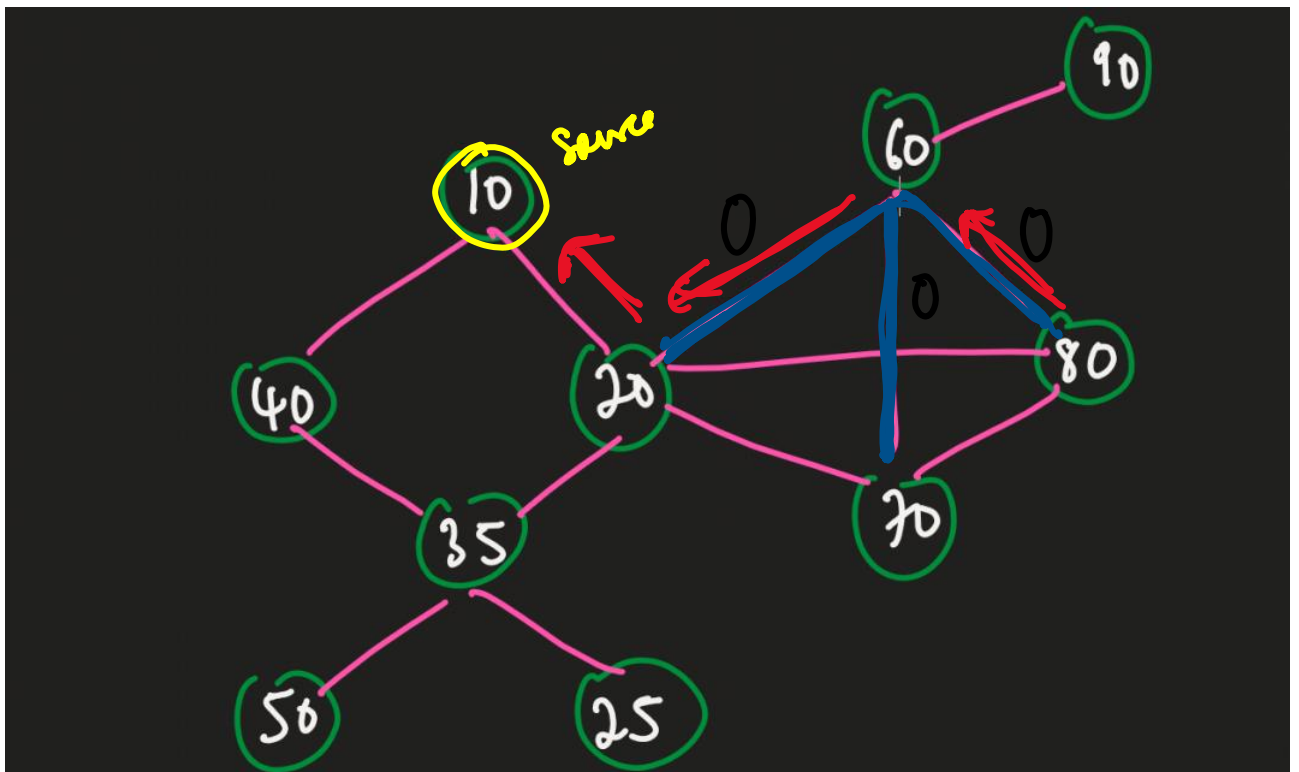
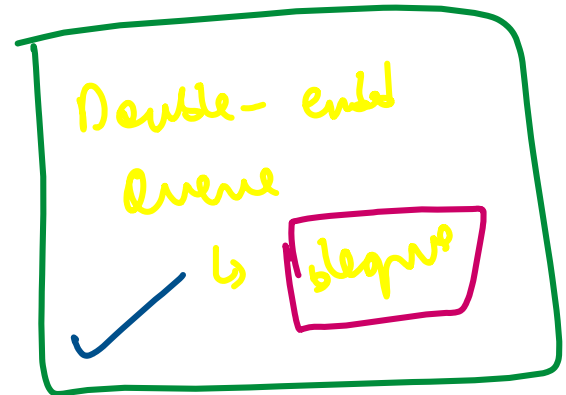
12:16 AM

What is 0-1 BFS?

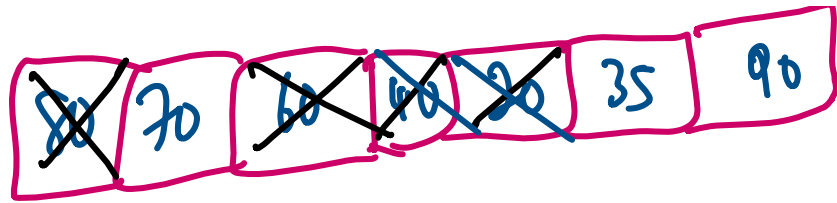
- Traversal algorithm which is used when the edge weights are only 0 or 1
- Most widely used to determine minimum cost / minimum distance / minimum operations

Intuition:

- Use a deque
- For each edge:
 - **Weight 0** → push to front
 - **Weight 1** → push to back
- This keeps nodes with lower cost processed first



Queue



distance {

10 : 0,

60 : ~~1~~

90 : ~~2~~

40 : ~~inf~~ 1,

70 : ~~2~~ 1,

80 : 2 1

20 : ~~inf~~ 1,

35 : ~~inf~~ 2,

curr $\Rightarrow$ 10

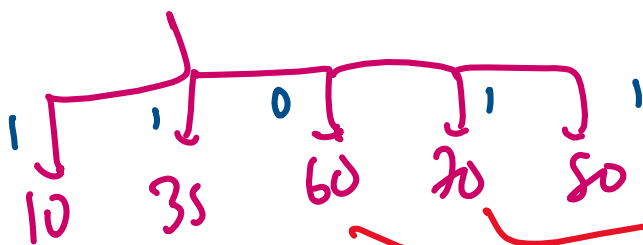


$$\left[\begin{array}{l} 0 + 1 < \text{inf} \\ \Rightarrow 1 < \text{inf} \end{array} \right] \quad \left[\begin{array}{l} 0 + 1 < \text{inf} \\ 1 < \text{inf} \end{array} \right]$$

$$\left[\begin{array}{l} 1 + 1 < 0 \\ 2 < 0 \end{array} \right]$$

$$\left[\begin{array}{l} 1 + 1 < 2 \\ 2 < 2 \end{array} \right]$$

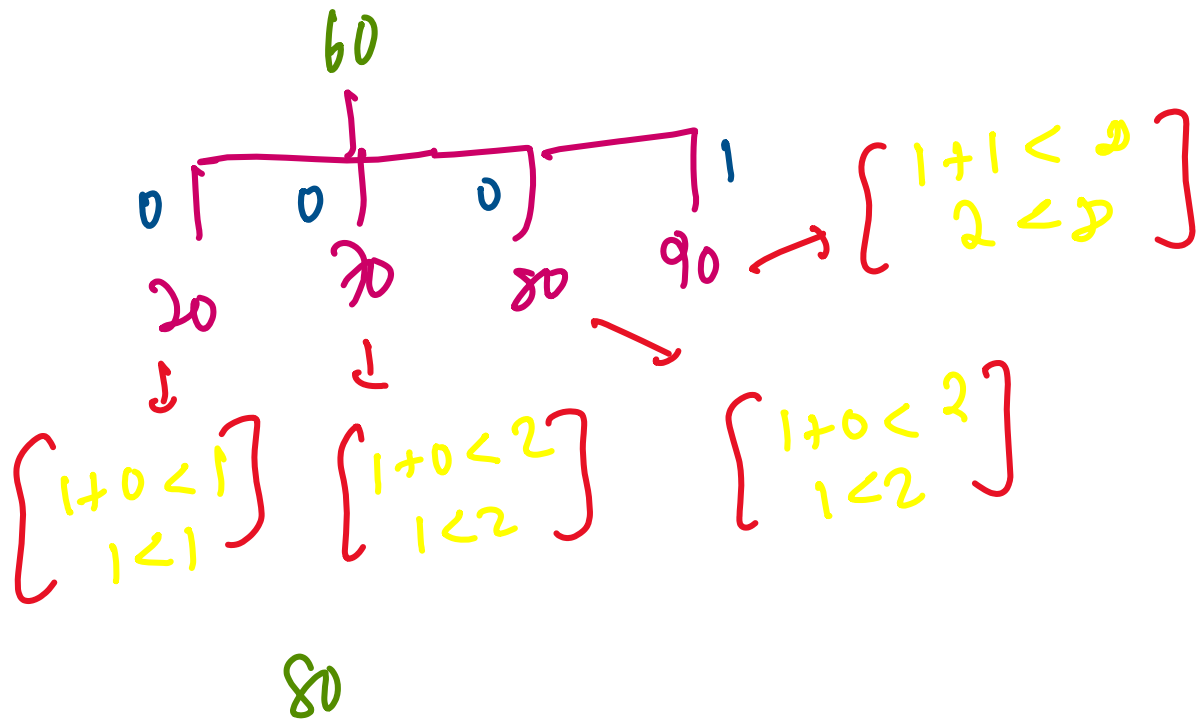
20



$$\left[\begin{array}{l} 1 + 1 < 2 \\ 2 < 2 \end{array} \right]$$

$$\left[\begin{array}{l} 1 + 1 < 0 \end{array} \right] \quad \left[\begin{array}{l} 1 + 1 < 2 \end{array} \right] \quad \left[\begin{array}{l} 1 + 0 < 2 \\ 1 < 0 \end{array} \right] \quad \left[\begin{array}{l} 1 + 1 < 2 \\ 2 < 2 \end{array} \right]$$

$$\left[\begin{array}{l} 1+1 < 0 \\ 2 < 0 \end{array} \right] \quad \left[\begin{array}{l} 1+1 < 2 \\ 2 < 2 \end{array} \right] \quad \left[\begin{array}{l} 1+0 < 2 \\ 1 < 8 \end{array} \right] \quad \left[\begin{array}{l} 2 < 8 \end{array} \right]$$



13.6.5 - Choice

Friday, January 2, 2026 1:08 AM

SCENARIO	ALGORITHM
Shortest path	BFS
Exhaust all paths	DFS
Cycle detection	DFS
Grid minimum steps	BFS
Backtracking	DFS

13.7 - Shortest Path Algorithms

Wednesday, December 31, 2025 12:16 AM

What is a Shortest Path?

The shortest path problem is about finding the **minimum cost path** between nodes in a graph

- Cost can be:
 - *Distance*
 - *Time*
 - *Money*
 - *Energy*
- Graph can be:
 - *Directed / Undirected*
 - *Weighted*

Most Popular Algorithms:

- Dijkstra's Algorithm
- Bellman–Ford Algorithm
- Floyd–Warshall Algorithm

13.7.1 - Dijkstra's

Thursday, January 1, 2026 11:59 PM

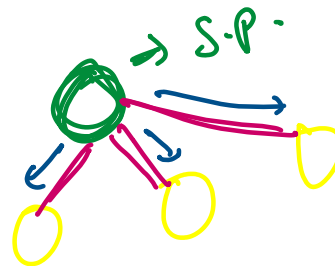
- Dijkstra's Algorithm is a shortest path algorithm used in graphs to find the minimum distance from a single source vertex to all other vertices
- It is one of the most important algorithms in DSA and graphs, especially for **weighted graphs**
- Does not work with negative weights
- Very fast for large and sparse graphs
- Helps compute the shortest path distance from the source to every other node, given:
 - **Graph**
 - **Source node**

Key Conditions:

- Graph can be **directed or undirected**
- Graph must have **non-negative edge weights (≥ 0)**

Intuition: **Always expand the nearest unvisited node first**

- Start from the source
- Update distances of neighbors
- Always pick the next node with the smallest known distance
 - This is why we use a **min-heap (priority queue)**



$$(0 + 0) \Rightarrow S.P.$$

Data Structures Used:

- **Adjacency List** → graph representation
- **Min Heap** → always get the closest node

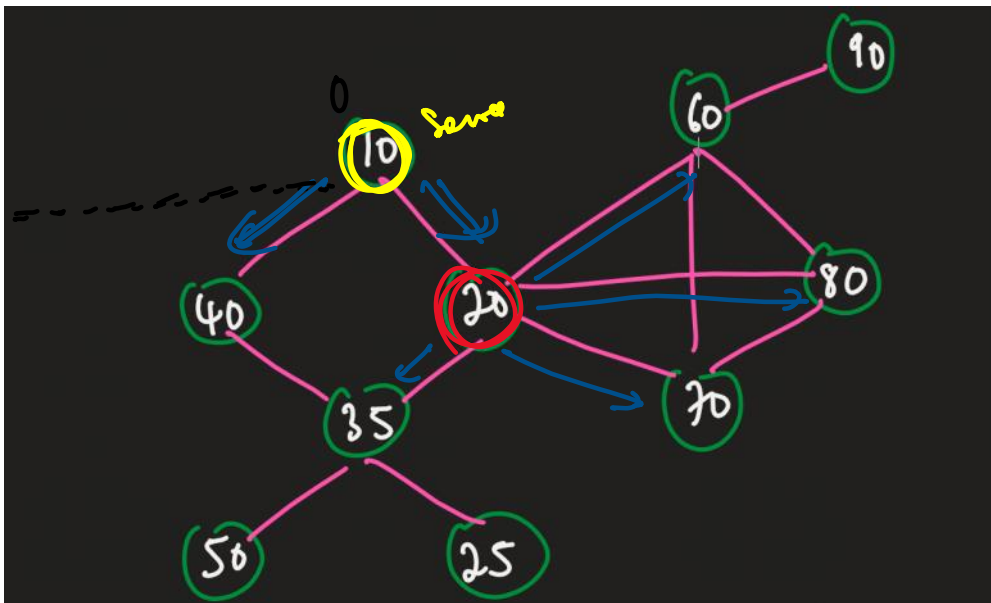
Algorithm:

1. Set distance of all nodes = ∞
2. Set source distance = 0
3. Push (0, source) into min-heap
4. Pop smallest distance node
5. Relax its neighbors
6. Repeat until heap is empty

} Initial
} Repeated

Complexity Analysis:

- **Time:** $O((V + E) \log V)$
- **Space:** $O(V)$



13.7.2 - Bellman-Ford

Thursday, January 1, 2026 11:59 PM

- Bellman–Ford Algorithm is a single-source shortest path algorithm used in graphs that can handle negative edge weights
- It is especially important because it can also detect negative weight cycles, which Dijkstra's algorithm cannot do
- Generally, its slower than Dijkstra's algorithm
- The shortest distance from the source to all other vertices, even when negative edges are present, given:
 - *Graph*
 - *Source node*

Key Features:

- Works with negative edge weights
- Works for directed and undirected graphs
- Can detect negative cycles

Intuition: Relax all edges repeatedly, $V-1$ times

- The shortest path between two nodes can have *at most $V-1$ edges* (where V = number of vertices)
- Each relaxation improves distances gradually
- If we relax all edges $V-1$ times, *shortest paths are guaranteed*

Data Structures Used:

- **Adjacency List** → graph representation
- **Distance Array** → store shortest paths

Algorithm:

- **Initialize:**
 - Distance of source = 0
 - Distance of all other vertices = ∞
- **Repeat $V-1$ times:**
 - For every edge ($u \rightarrow v$, weight w)
- **Negative cycle check:**
 - Do one more relaxation

- If any distance still reduces $\rightarrow$ **negative cycle exists**

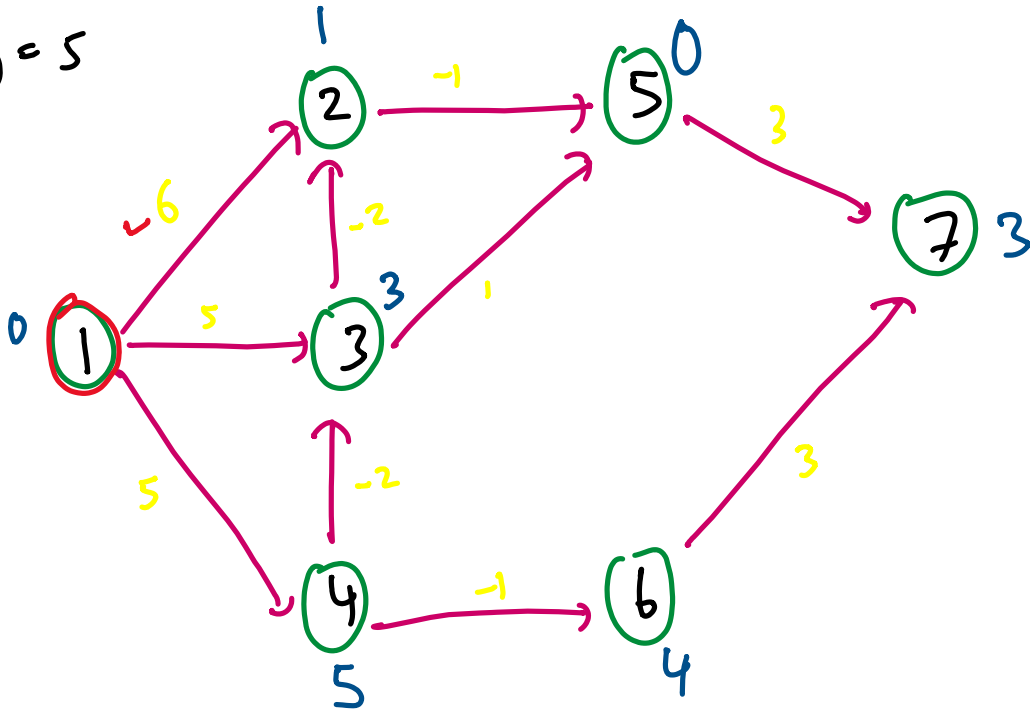
Complexity Analysis:

- Time: $O(V * E)$
- Space: $O(V)$

$$|V| = 7$$

$$\Rightarrow V - 1 \rightarrow \underline{6}$$

$$6 + (-1) = 5$$



||||

$$\text{node}(u) + w < \text{node}(v) :$$

$$\text{node}(v) = \text{node}(u) + w$$

13.7.3 - Floyd-Warshall

Thursday, January 1, 2026 11:59 PM

- Floyd–Warshall Algorithm is an all-pairs shortest path algorithm used in graphs to find the shortest distance between every pair of vertices
- It is especially useful for dense graphs and graphs with negative edge weights (but no negative cycles)
- Works best when the graph is **small or dense**
- Works even if negative edges exist

Key Features:

- Computes all-pairs shortest paths
- Works with negative edge weights

Intuition: Try every vertex as an intermediate point between every pair of vertices

- Try every vertex as an intermediate point between every pair of vertices

Data Structures Used:

- **Adjacency List** → graph representation
- **Distance Array** → store shortest paths

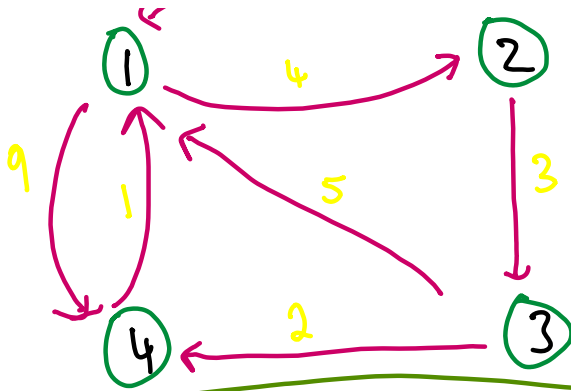
Algorithm:

- **Create a Distance Matrix:**
 - $\text{dist}[i][j] = \text{weight}(i \rightarrow j)$
 - 0 if $i == j$
 - ∞ if no edge exists
- **For each vertex k, i, j:**
 - $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$
- **Final matrix contains shortest distances between all pairs**

Complexity Analysis:

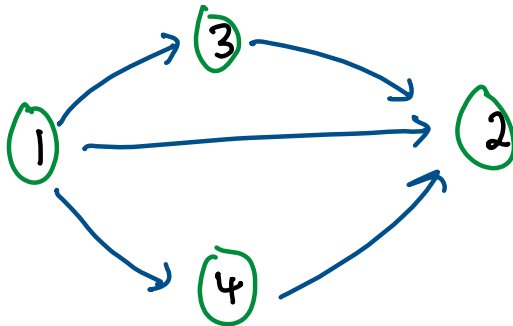
- **Time:** $O(V * V * V)$
- **Space:** $O(V * V)$





$A^0 =$

	1	2	3	4
1	0	4	∞	9
2	10	0	3	∞
3	5	∞	0	2
4	1	∞	∞	0



i) Consider ① as intermediate vertex

$A^1 =$

	1	2	3	4
1	0	4	∞	9
2	10	0	3	19
3	5	9	0	2
4	1	5	∞	0

$$\min(\infty, 10+9)$$

$$\min(\infty, 19)$$

$$\min(\infty, 5+4) \rightarrow \min(\infty, 9)$$

$$\min(2, 5+9) \rightarrow (2, 14)$$

$$\min(\infty, 1+2) \rightarrow \min(\infty, 3)$$

1, 2, 3, 4

$$\min(\infty, 1+2)$$

$$A^1[2,3] = \min \left(A^0[2,3], A^0[2,1] + A^0[1,3] \right)$$

$$\min(2, 1+2)$$

$$A^k[i,j] = \min \left(A^{k-1}[i,j], A^{k-1}[i,k] + A^{k-1}[k,j] \right)$$

$$\min(3, 10+2) \rightarrow (3, 2)$$

$$A^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 4 & 7 & 9 \\ 10 & 0 & 3 & 19 \\ 5 & 9 & 0 & 2 \\ 1 & 5 & 8 & 0 \end{bmatrix} \end{matrix}$$

$$\min(2, 4+3) \rightarrow (2, 7)$$

$$\min(9, 4+19) \rightarrow (9, 23)$$

$$\min(5, 10+9) \rightarrow (5, 19)$$

$$\min(2, 9+19) \rightarrow (2, 28)$$

$$\min(1, 15)$$

$$\min(2, 5+3) \rightarrow \min(2, 8)$$

$$A^3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 4 & 7 & 9 \\ 8 & 0 & 3 & 5 \\ 5 & 9 & 0 & 2 \\ 1 & 5 & 8 & 0 \end{bmatrix} \end{matrix}$$

$$\min(4, 7+9) \rightarrow \min(4, 16)$$

$$\min(9, 9)$$

$$\min(10, 8) \rightarrow 8$$

$$\min(19, 5) \quad \min(5, 17)$$

4 | 1 5 8 0

$\min(10, 0) = 0$

$\min(19, 5)$

$\min(5, 17)$

$\min(1, 13)$

$A_4 =$

	1	2	3	4
1	0	4	7	9
2	6	0	3	5
3	3	7	0	2
4	1	5	8	0

$\min(4, 14)$

$\min(7, 17)$

$\min(8, 6)$

$\min(3, 13)$

$\min(5, 3)$

$\min(9, 7)$

13.8 - Advanced

Saturday, January 3, 2026 12:25 AM

- **Disjoint Set Union (Union–Find)**
- **Kruskal's Minimum Spanning Tree (MST)**
- **Prim's MST**

13.8.1 - DSU

Saturday, January 3, 2026 12:26 AM

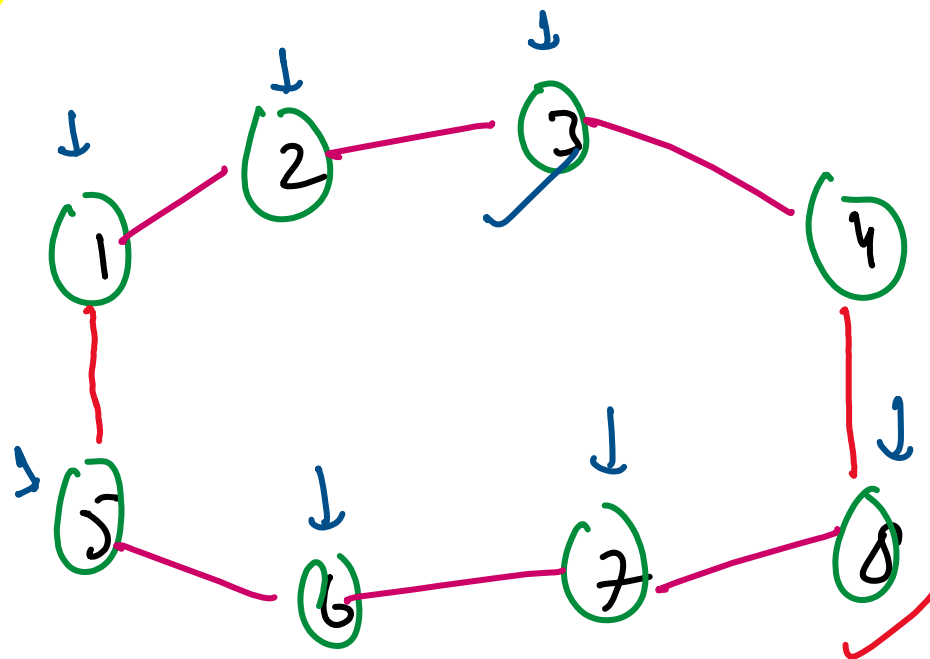
What is DSU?

- DSU is a data structure that keeps track of **connected components** dynamically
- **Used when:**
 - Graph edges are added gradually
 - Fast cycle detection
 - Check connectivity

Key Operations:

- Union
- Find

$$S_1 = \{1, 2, 3, 4\}$$
$$S_2 = \{5, 6, 7, 8\}$$



Find:

- * The element will belong only single set
- * Search set by set

$$S1 \cap S2 = \emptyset$$

$$S3 = \{1, 2, 3, 4, 5, 6, 7, 8\}$$

$$(1, 5) \rightarrow S3$$

∴ There's a cycle in graph

$$\{0, 1, 2\}$$

$$\{3, 4\}$$

$\{0, 1, 2\}$

S_1

$\{3, 4\}$

S_2

What is Prim's MST:

- Prim's Algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph
- Suitable to use when:
 - *Graph is connected*
 - *Graph is undirected*
 - *Graph has weights*
 - *Best for dense graphs*

What is MST?

A spanning tree that:

- *Connects all vertices*
- *Has no cycles*
- *Has minimum total edge weight*

What is a Spanning Tree:

A spanning tree is a subgraph of a connected graph that:

- *Includes all vertices*
- *Has no cycles*
- *Uses minimum number of edges*

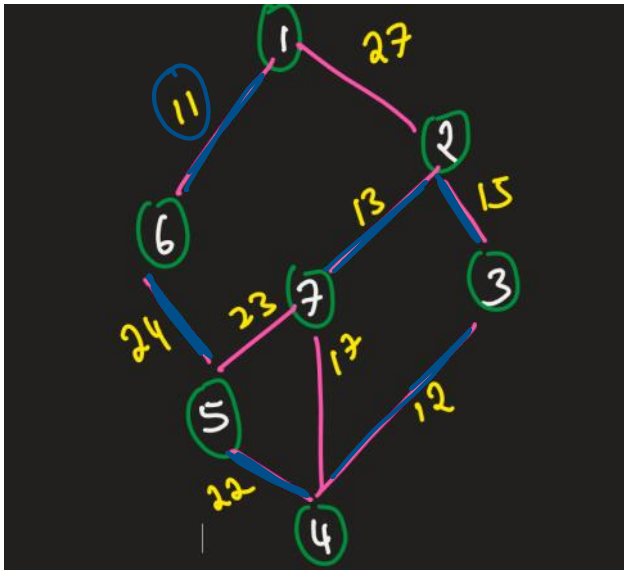
Intuition:

- Grow the MST one vertex at a time, always choosing the cheapest edge that connects:
 - *a vertex inside the MST*
 - *to a vertex outside the MST*
- **Analogy:** Expanding a network step-by-step, always picking the **lowest-cost connection available**

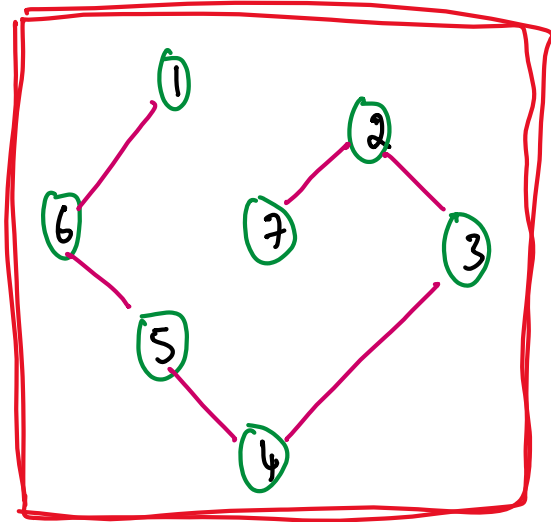
Algorithm:

- Pick any start vertex
- Mark it as visited (part of MST)
- Push all its edges into a **min-heap (priority queue)**
- **Loop:**
 - Extract the **minimum weight edge**
 - If the destination vertex is not visited:
 - Add edge to MST
 - Mark vertex as visited
 - Push its edges into heap
 - Stop when MST has $V - 1$ edges

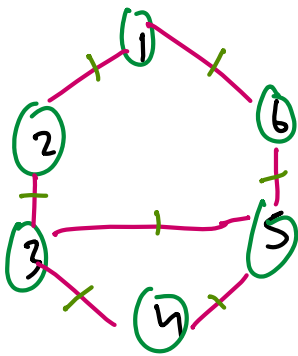




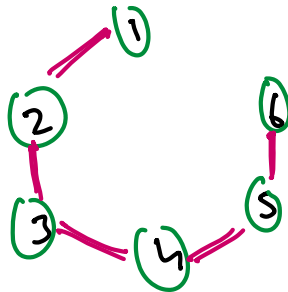
27, 24
22, 23,
2, 17, 15
13



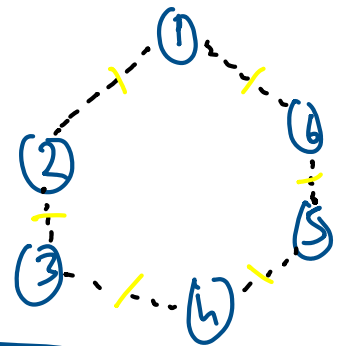
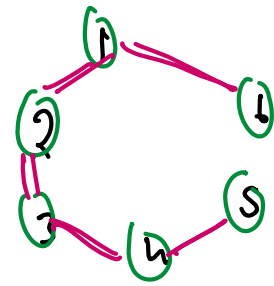
Total Cost = 97



$|V| = 6$
 $|E| = 7$



$|V'| = |V| = 6$
 $|E'| \neq |E|$
 $|E'| < |E|$
 $|E'| = 5$



$6 \subset 5$

13.8.3 - Kruskal's MST

Saturday, January 3, 2026 12:31 AM

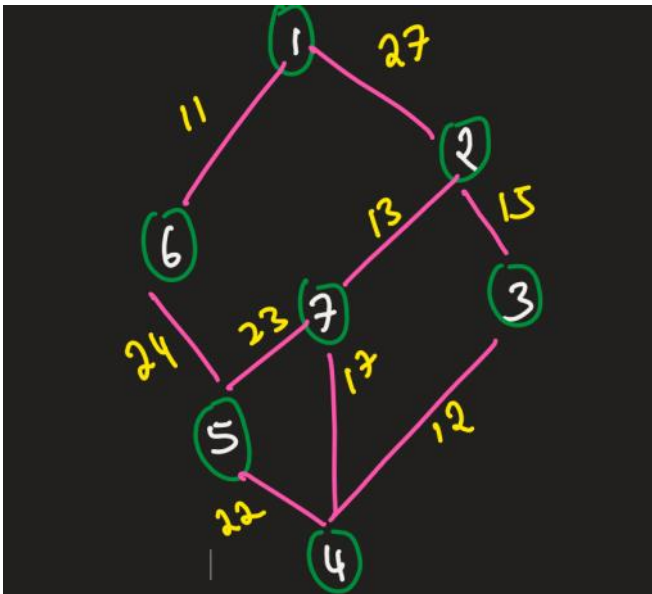
Algorithm:

1. Sort edges by weight
2. Pick smallest edge that does not form a cycle
3. Stop after $V - 1$ edges

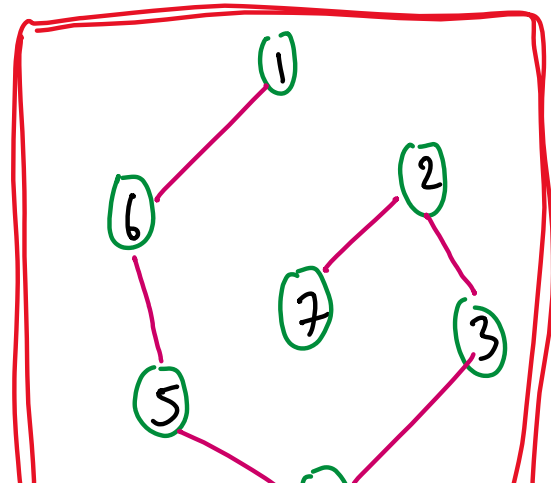
Note: Cycle detection is done using DSU

When to Use Kruskal?

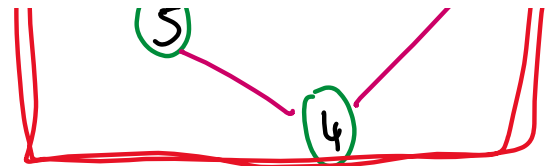
- Sparse graphs
- Edge list available
- Offline processing



	u	v	w	
✓	1	6	11	11
✓	4	3	12	23
✓	2	7	13	36
				51



✓	2	7	15	26
✓	2	3	15	51
✗	4	7	17	
✓	4	5	22	73
✗	5	7	23	
✓	5	6	24	97
	1	2	27	



 Total Cost = 97

13.8.4 - Prim vs Kruskal

Saturday, January 3, 2026 12:40 AM

FEATURE	PRIM	KRUSKAL
Approach	Vertex-based	Edge-based
Data Structure	Min Heap	DSU (Union-Find)
Best for	Dense graphs	Sparse graphs
Graph type	Connected	Can handle disconnected (forest)



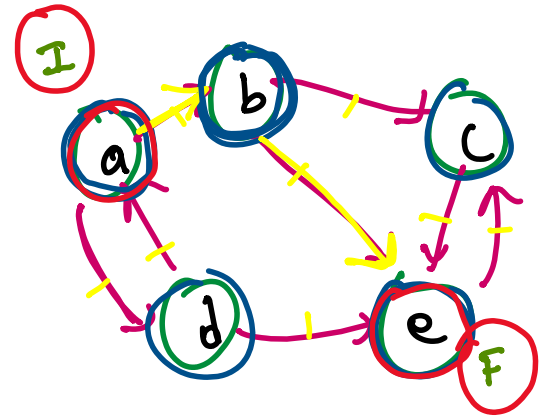
13.9 - State Space

Saturday, January 3, 2026 12:58 AM

What is a State Space Graph?

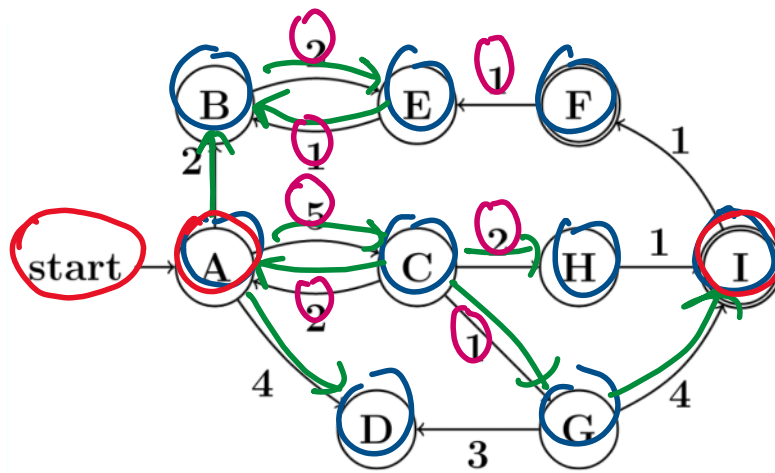
A State Space Graph is a graph where:

- Each node represents a state
- Each edge represents a valid transition (action)
- We search the graph to reach a goal state from an initial state



A State Space Graph is defined as:

- $G = (S, A)$
- S = set of all possible states
- A = set of actions that move from one state to another



Components of a State Space Graph:

1. Initial State:

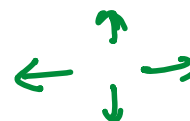
- The position where the problem starts

2. State:

- A state captures all information needed to continue the problem

3. Actions / Transitions:

- Valid moves from one state to another
- Move Up, Down, Left, Right, etc.



4. Goal State:

- The state that solves the problem

5. Constraints:

- Rules that decide whether a state is **valid or invalid**

How State Space Graphs Are Explored:

- ✓ • **BFS:** *Shortest path / minimum steps*
- ✓ • **DFS:** *Exploration*
- ✓ • **Backtracking:** *Constraint satisfaction*
- ✓ • **Dijkstra / A*:** *Weighted state transitions*

When to Use State Space Graph Modeling?

- The problem has choices at each step
- Need to explore all possible configurations
- Constraints decide validity
- BFS/DFS/Backtracking fits naturally

Complexity Analysis:

- **Time:** Depends on the number of states
- **Space:** Depends on visited states / recursion
- **Worst Case:** Exponential

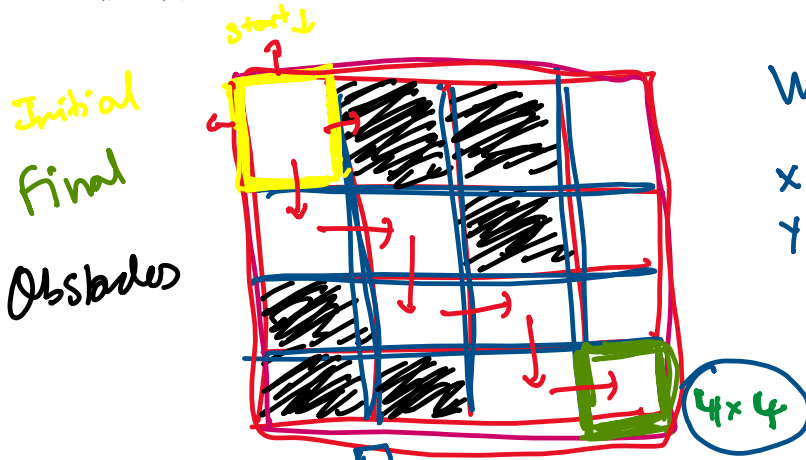
Real-World Applications:

- **AI Search Problems**
- **Game Solvers (Chess, Sudoku)**
- **Robot Navigation**
- **Path Planning**

- **Compiler optimization**

13.9.1 - Rat in a Maze

Saturday, January 3, 2026 10:06 AM

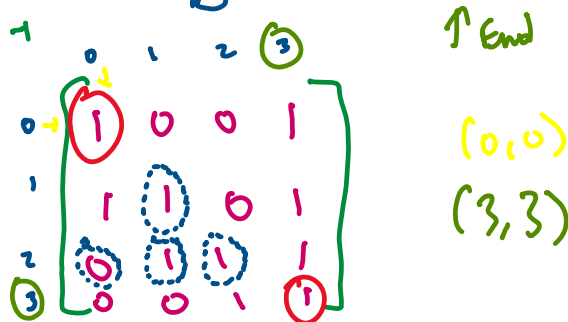


Wanted = $\{(0,0), (1,0), (1,1), (2,1), (2,2), (3,2), (3,3)\}$

$x = 0, 1, 2, 3$
 $y = 0, 1, 2, 3$

$x = m-1$
 $y = n-1$

(2,1)



m → rows
n → columns

States: Cells having 1

Constraints: position must be within (0,0) & (m-1, n-1)

Actions: up, down, right, left

$(-1,0)$ $(1,0)$ $(0,1)$ $(0,-1)$

Start → (0,0)

$maze[0,0] = 1$

(x,y)

Queue = [(0,0),

Visited = {(0,0),

for (dx, dy) in options:

nx, ny = (x+dx), (y+dy)

if $(0 \leq nx < m) \& (0 \leq ny < n) \& \text{maze}[nx][ny] == 1$:

visited.add((nx, ny))

enqueue((nx, ny))

while queue:

curr = dequeue()

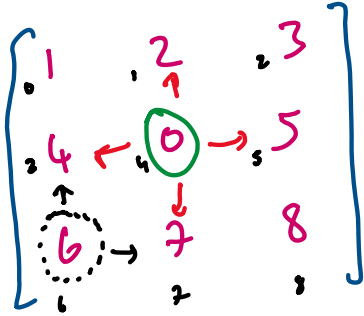
if $x, y == (m-1), (n-1)$:

True

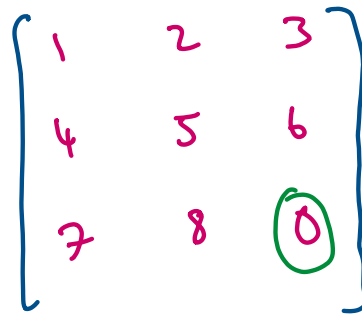
keep our actions

return false

Initial State:



Final State:

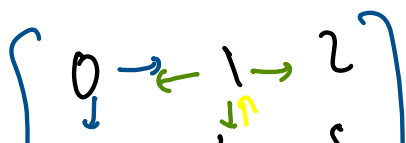
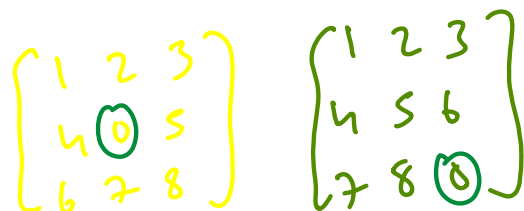


$$\begin{bmatrix} 0 \\ 4 \end{bmatrix} \rightarrow \begin{bmatrix} 4 \\ 0 \end{bmatrix}$$

Swap

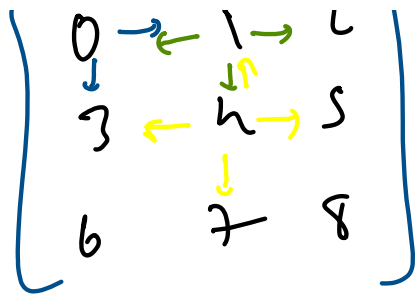
Arrangement of tiles (0-8)

Move tile '0' from initial to goal state

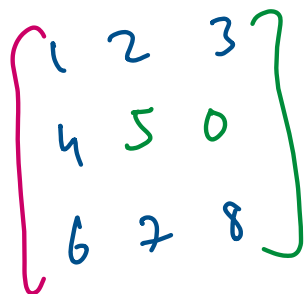
Represent states as tuplesMoving tile $\rightarrow$ SwappingInitial $\rightarrow (1, 2, 3, 4, 0, 5, 6, 7, 8)$ Goal $\rightarrow (1, 2, 3, 4, 5, 6, 7, 8, 0)$ 

Swaps = $\{$

0: $[1, 3]$, 1: $[0, 2, 4]$,



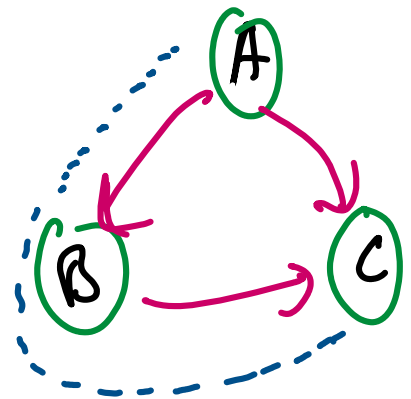
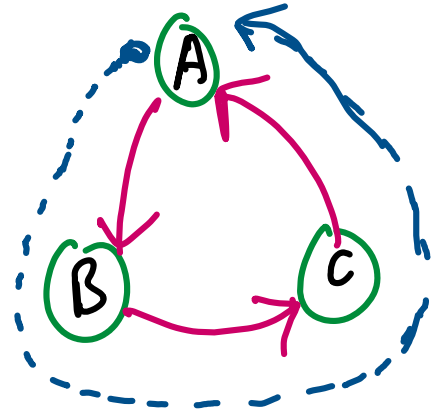
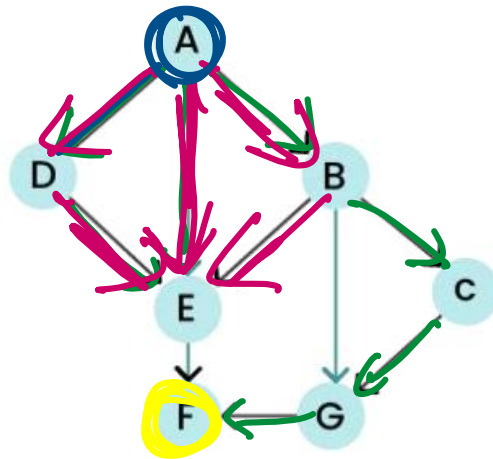
0: [1, 3], 1: [0, 2, 4],
 2: [1, 5],
 4: [1, 3, 5, 7]
 }



What is a DAG?

A Directed Acyclic Graph (DAG) is a graph which is:

1. **Directed:** edges have direction
2. **Acyclic:** no cycles



Key Properties of DAG:

- At Least One Topological Ordering Exists
- At Least One Node with In-degree = 0
- At Least One Node with Out-degree = 0
- Longest Path Can Be Computed Efficiently
- Multiple Topological Orders May Exist
- Represents Dependency Relationships
- Every Subgraph of a DAG Is also a DAG
- Cycle Detection Is Simple

Real-World Applications:

- **Task / Job Scheduling:**
 - Tasks depend on other tasks
 - A task can start only after its dependencies finish
 - Used in *Project management tools, Workflow Engines*
- **Build Systems (Makefiles):**
 - Some files must be compiled before others
 - Used in *CI/CD pipelines*
- **Dependency Resolution:**
 - Used in software development
 - *Docker image layers*
- **Data Pipelines & ETL Workflows:**
 - Data flows through **multiple processing stages**
 - *Apache Airflow, Luigi, Prefect*
- **Version Control Systems:**
 - Commit history is a DAG
 - Each commit points to parent commit(s)
- **Neural Networks & Computation Graphs:**
 - Operations depend on outputs of other operations
 - *Tensorflow, Pytorch*

Most Popular DAG Algorithms:

- **Topological Sorting (Kahn's Algorithm)**
- **Shortest Path in DAG**
- **Longest Path in DAG**

- **Counting Paths in DAG**

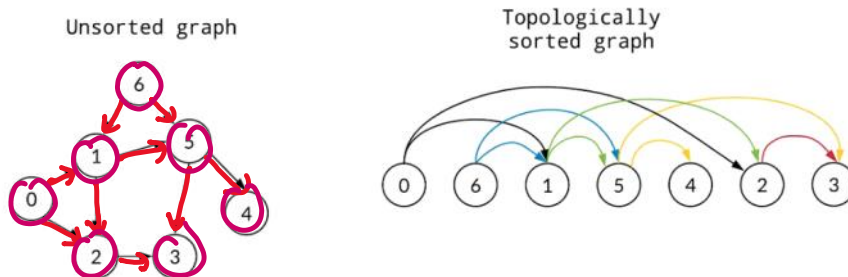
13.10.1 - Kahn's Algorithm

Saturday, January 3, 2026 10:47 AM

- Kahn's Algorithm finds a topological order of a DAG by repeatedly removing nodes with no dependencies
- If this process cannot remove all nodes, the graph contains a cycle

Topological Ordering:

- Linear arrangement of vertices in a DAG where for every directed edge from vertex u to v , u always appears before v in the list
- This shows dependency order like task prerequisites or build processes
- Allows representing workflows, ensuring that a task is listed only after its dependencies are met



Main Ideas:

1. Compute **in-degree** of every node
2. Push nodes with **in-degree = 0** into queue
3. Remove them and reduce neighbors' in-degree

Conclusions:

- If at end, all nodes processed → **DAG**
- Else → **cycle exists**

Algorithm:

- Find all nodes with **in-degree = 0**
- Put them in a queue
- Process nodes level by level (BFS)
- Reduce in-degree of neighbors
- Add new **zero in-degree** nodes to queue

- Add new *zero in-degree* nodes to queue

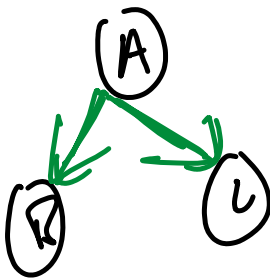
in-degree = {
 $0: 0$, $1: \cancel{2}^0$, $2: \cancel{2}^0$,
 $3: \cancel{2}^0$, $4: \cancel{1}^0$, $5: \cancel{2}^0$,
 $6: 0$
 }

$t = \{\underline{0}, \underline{6}, \underline{1}, \underline{2}, \underline{5}, \underline{3}, \underline{4}\}$

queue: {0, 6, 1, 2, 5, 3, 4}

$u = 0\ 6\ 1\ 2\ 5\ 3\ 4$

DAG



$A: [B, C]$

13.10.2 - Shortest Path

Saturday, January 3, 2026 10:48 AM

Properties:

1. Handles Negative Edge Weights
2. **Faster Than Other Algorithms** (Bellman, Dijkstra)

Algorithm:

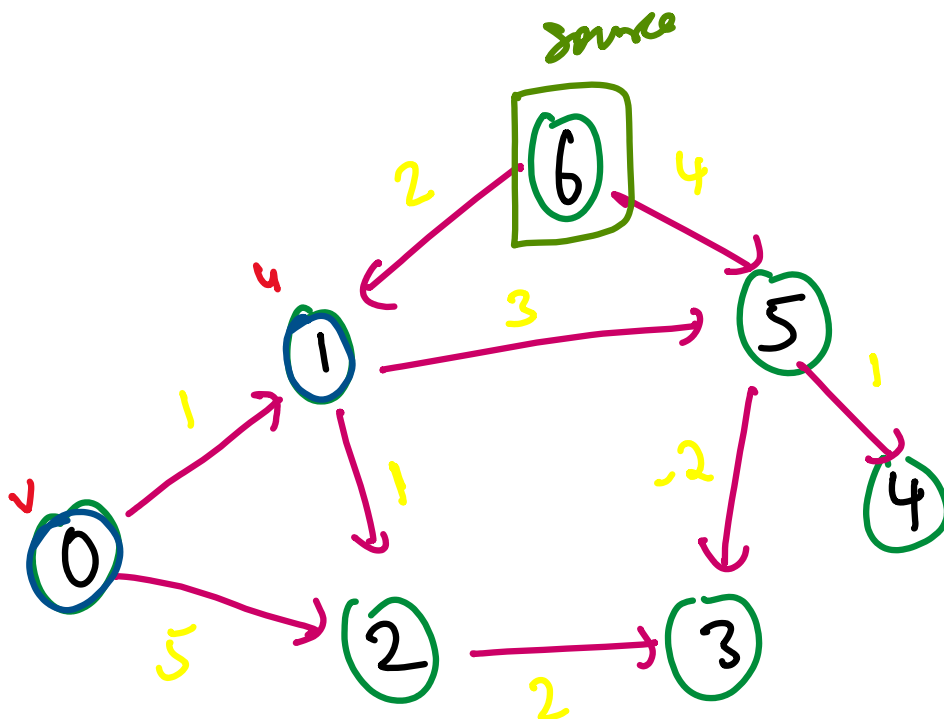
1. Topologically Sort the Graph:

- Ensures correct dependency order
- Any valid topological order works

2. Initialize Distances:

- Set distance of source = 0
- Set all other nodes = ∞

3. Relax Edges in Topological Order



$sp[1]$

$sp[0] \rightarrow ?$

$$\underbrace{sp[0]} > \underbrace{sp[1] + 1}_{\text{shorter path}}$$

$$sp[0] = sp[1] + 1$$

NOTE: Shortest path to node (0)
will remain ∞ because it
has no incoming node

13.10.3 - Longest Path

Saturday, January 3, 2026 10:48 AM

Properties:

- Works with **positive, zero, and negative weights**
- Based on **topological sorting**
- Very similar to shortest path in DAG, just with **max** instead of **min**

13.10.4 - Counting Paths

Saturday, January 3, 2026 10:48 AM

What Is the Counting Paths Problem?

- Given a DAG and a source node **S**, count the number of distinct paths from **S** to every other node (or to a specific target **T**)
- Paths must follow edge direction
- Nodes cannot repeat (because DAG has no cycles)

Properties:

- Only works correctly for DAG
- Uses topological sorting
- Each node's path count is finalized once
- Linear time complexity
- **Key Idea:** Number of ways to reach a node = sum of ways to reach its parents

Algorithm:

1. Topologically Sort the DAG:

- Ensures all incoming paths are counted before processing a node

2. Initialize Path Counts

3. Propagate Path Counts

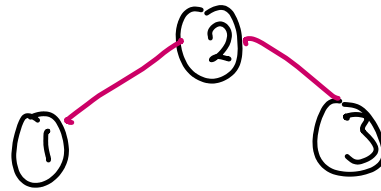
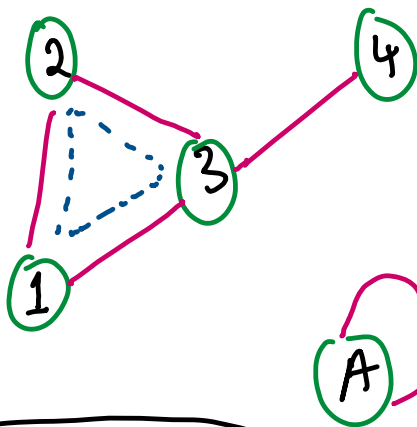
13.10.5 - Summary

Saturday, January 3, 2026 11:46 AM

FEATURE	Kahn's Algorithm	Shortest Path in DAG	Longest Path in DAG	Counting Paths in DAG
Purpose	Find a linear order of vertices respecting dependencies	Find minimum distance/cost from source to all vertices	Find maximum distance/cost from source to all vertices	Count number of distinct paths from source to target
Input	Directed Acyclic Graph (DAG)	Weighted DAG (can be negative)	Weighted DAG (can be negative)	DAG
Output	Order of vertices	Shortest distance to each vertex	Longest distance to each vertex	Number of paths to each vertex
Key Technique	BFS + in-degree tracking (Kahn)	Topo-sort + edge relaxation	Topo-sort + edge relaxation (maximize)	DP on topologically sorted vertices
Time Complexity	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V + E)$
Real-world Use Cases	Task scheduling, build systems, course planning	Project scheduling, routing, CPU instruction scheduling	Critical path analysis, bottleneck detection, project duration	Workflow analysis, counting decision paths, network reliability

13.11 - Coding Problems

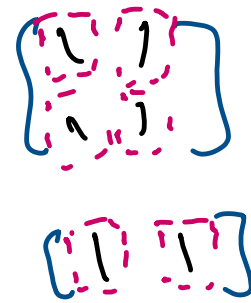
Saturday, January 3, 2026 11:58 AM



Hand-drawn DP table for a sequence of length 5 (n=5) and width 4 (m=4). The table is a 5x5 grid with rows indexed 0 to 4 and columns indexed 0 to 4. The value at (i,j) is 1 if there is a path from (0,0) to (i,j) and 0 otherwise. The paths are highlighted with yellow dashed boxes.

	0	1	2	3	4
0	1	1	0	0	0
1	1	1	0	0	0
2	0	0	1	0	0
3	0	0	0	1	1

Coordinates: (0,0) at top-left, (3,4) at bottom-right.



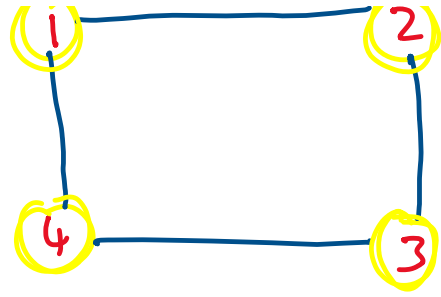
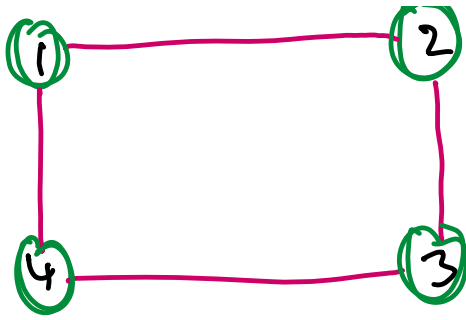
(3)

$m=4$
 $n=5$
 $(0,0)$
 $(m-1, n-1)$

up $\rightarrow (-1, 0)$
 down $\rightarrow (1, 0)$
 left $\rightarrow (0, -1)$
 right $\rightarrow (0, 1)$

$(1,1)$
 $\uparrow$
 $(2,0) \leftarrow (2,1) \rightarrow (2,2)$
 $\downarrow$
 $(3,1)$



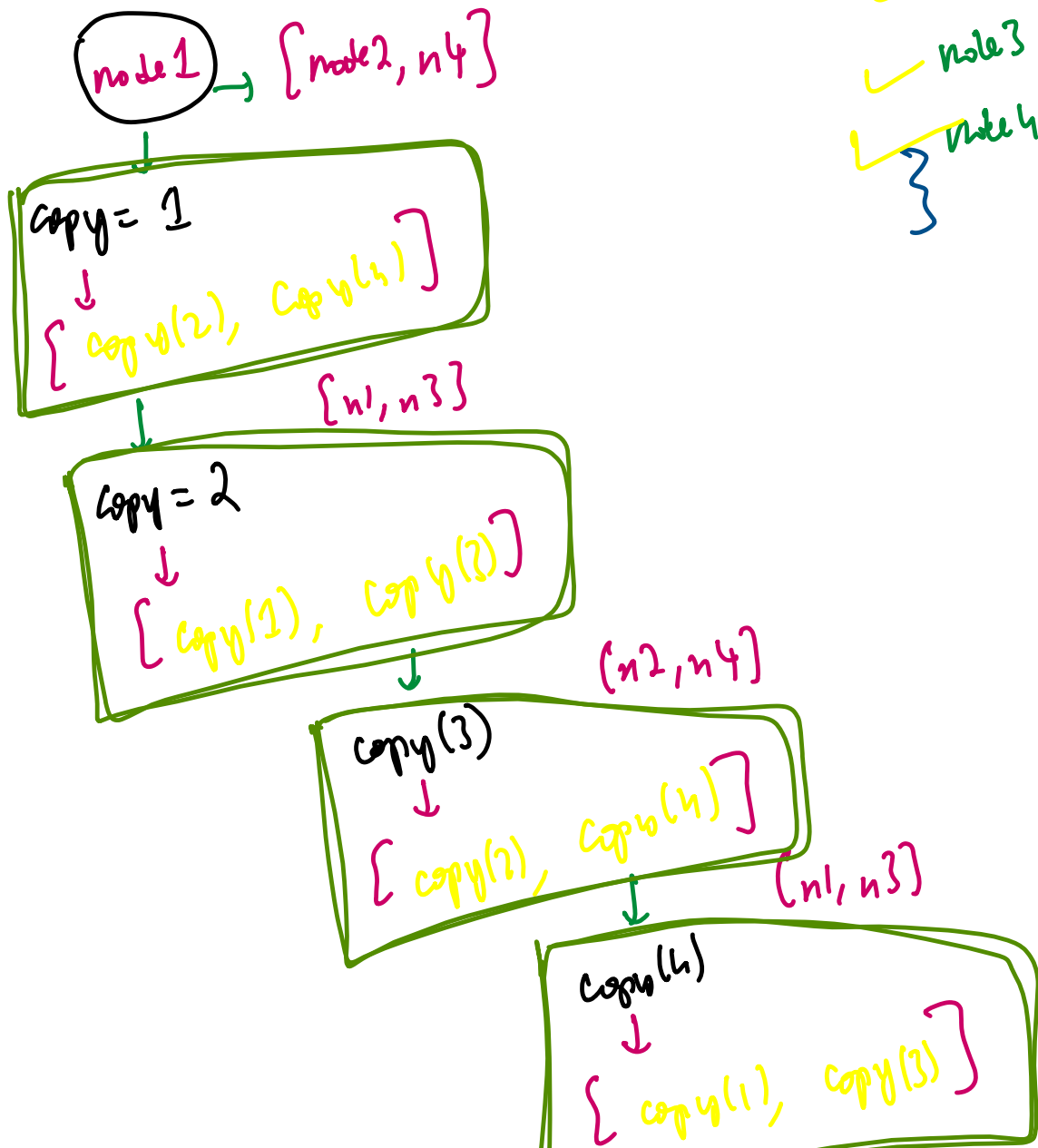


graph1 = graph2 ✓

graph1 is graph2 ✗

Map = {

- ✓ node1: copy1,
- ✓ node2: copy2,
- ✓ node3: copy3,
- ✓ node4: copy4



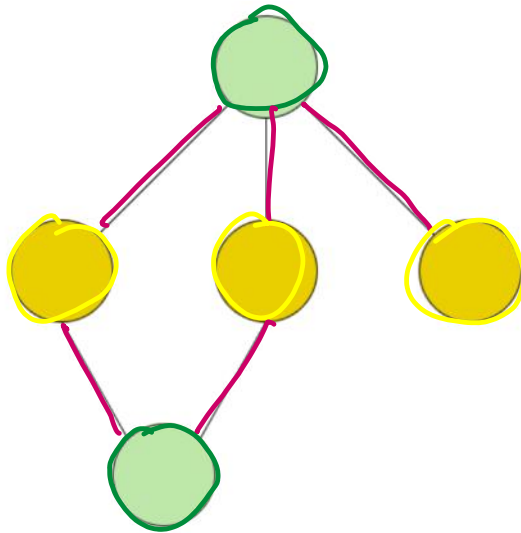
$\{ \text{copy}(1), \text{copy}(15) \}$

Bipartite Graph:

- Its vertices can be divided into two sets such that
- No two adjacent vertices are in the same set
- The graph can be colored using 2 colors
- No edge connects vertices of the same color

Idea:

- Assign a color (0 or 1) to a node
- All its neighbors must get the opposite color
- If a conflict appears $\rightarrow$ **not bipartite**



colors $\rightarrow \{0, 1\}$

color switch: 1-0

$\hookrightarrow 0 \rightarrow 1$
 $1 \rightarrow 0$

$\text{all}(T, T) \rightarrow T$
 $\text{all}(T, F) \rightarrow F$
 $\text{all}(F, F) \rightarrow F$

$1 \rightarrow 0$

num_Courses $\rightarrow$ int

prerequisites $\rightarrow$ list of lists

$4 \rightarrow 0, 1, 2, 3$

$\{1, 0\}, \{3, 2\}, \{2, 1\}$

DAG

Kahn's Algorithm



(sr, sc)

color

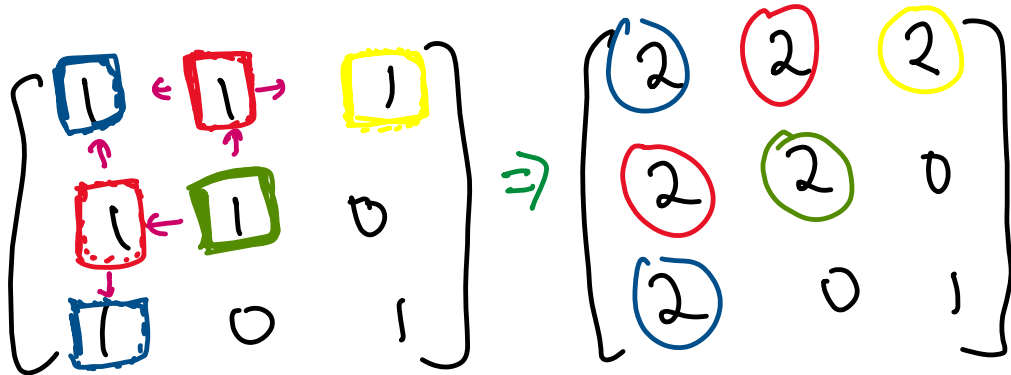
1. set (sr, sc) to color
2. set connected pixels to color

2. Set connected pixels to color
which one of some original color

$(sr, sc) \Rightarrow (1, 1)$

connectivity $\Rightarrow$ up, down, left, right

color $\Rightarrow 2$



cell $\rightarrow$ node

connection $\rightarrow$ edge

visited $\rightarrow$ traversal (DFS/BFS)

m, n
 $\downarrow \hookrightarrow 0, 1, 2, \dots, n-1$
 $0, 1, 2, \dots, m-1$

↓
0, 1, 2, ..., M-1

(1, 0), (-1, 0), (0, -1), (0, 1)

$R_1 \rightarrow [R_2]$

R_1, R_2, R_3

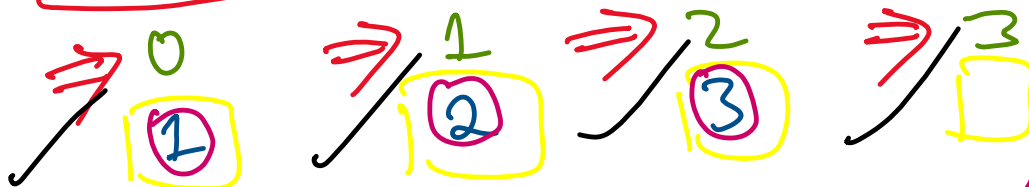
$R_2 \rightarrow [R_3]$

$R_3 \rightarrow []$

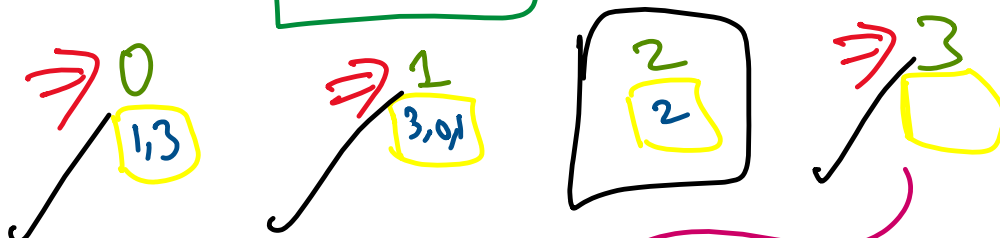
1. visit the room

2. Explore the keys

3. If not visited, visit that room

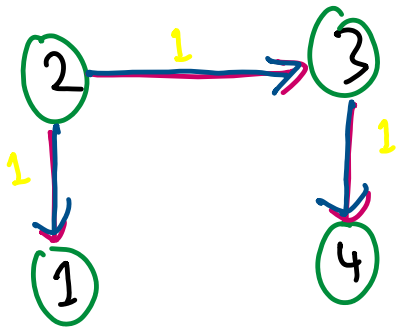


TRUE



FALSE

FALSE



Given:

1. Network of nodes
2. # nodes (n)
3. Starting node (k)

(u, v, w)

Minimum time taken by signal to reach all nodes

If signal can't reach all nodes
return -1

graph = {

1: [(2, 1), ()],

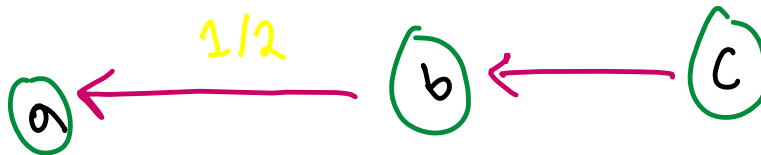
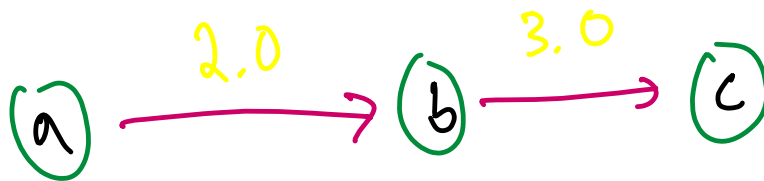
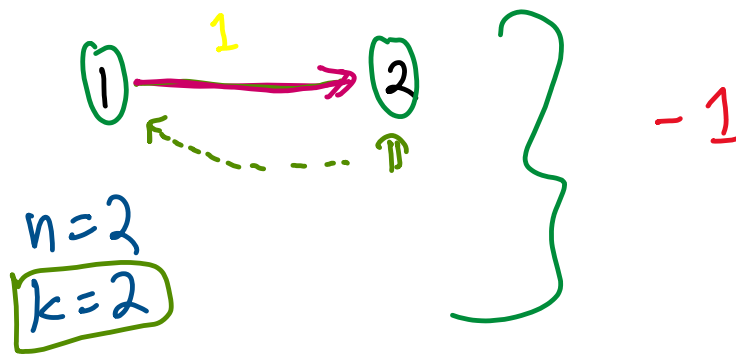
2: [],

3: [],

4: []

}

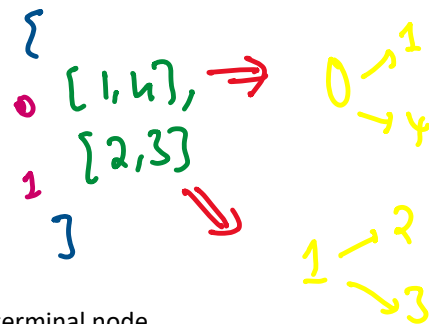
n 1 n 1



Eventual Safe States:

You are given a directed graph where:

- `graph[i]` = list of nodes you can go to from node `i`

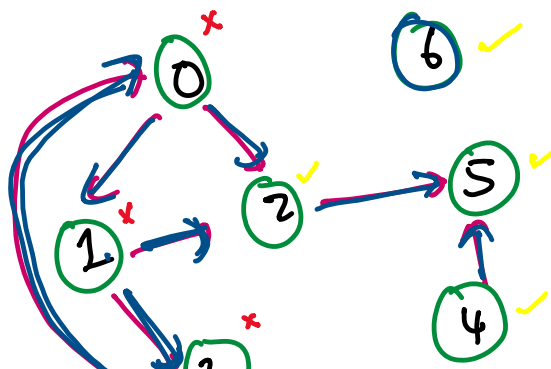


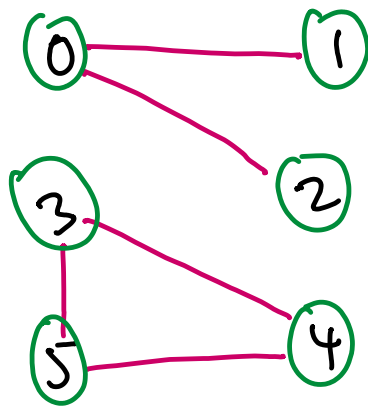
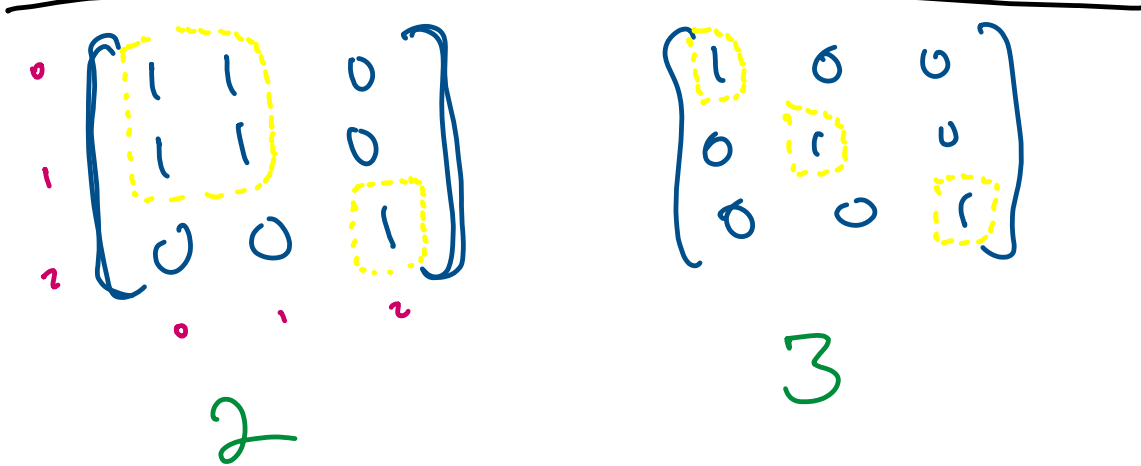
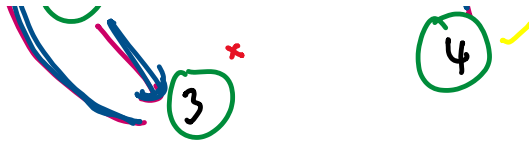
Safe Node:

- Starting from that node, every possible path eventually ends at a terminal node
- A terminal node has no outgoing edges

Unsafe Node:

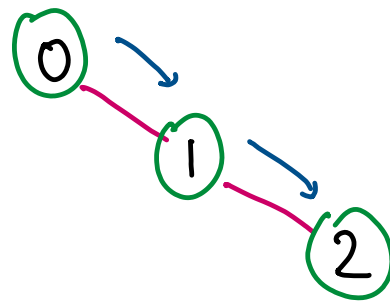
- A node that is part of a cycle
- Can reach a cycle





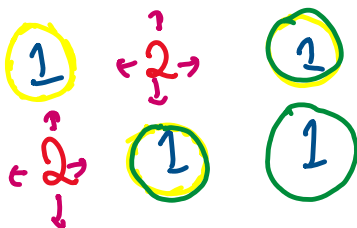
$s=0, d=5$

False



$s=0, d=2$

True



* Store the ~~set~~ oranges

* store the ~~status~~ oranges

* store the count of fresh oranges

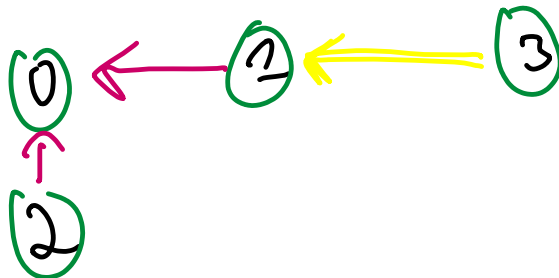
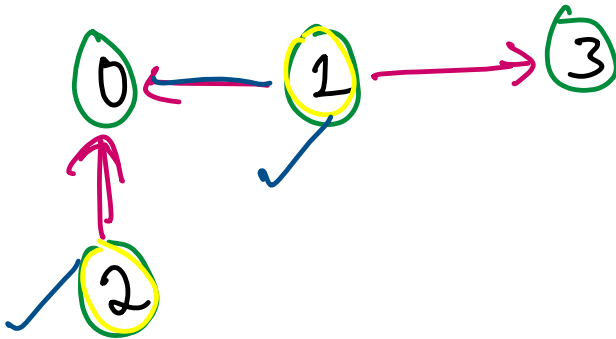
cell $\rightarrow$ node

adjacency $\rightarrow$ edges

* BFS

* multi-source BFS

* queue to store ~~status~~ oranges



13.12 - DS Context

Wednesday, January 7, 2026 12:48 AM

1. Recommendation Systems:

Graph Modeling

- Nodes: Users, Items
- Edges: Click, view, purchase, rating

ML Tasks

- Link prediction
- Node embeddings
- Personalized ranking

Algorithms Used

- Random Walks
- Node2Vec
- Graph Neural Networks (GNNs)

Real-World Examples

- Netflix movie recommendations
- Amazon product suggestions
- Spotify song recommendations

2. Fraud / Anomaly Detection:

Graph Modeling

- Nodes: Accounts, cards, users
- Edges: Transactions, transfers

ML Tasks

- Node classification (fraud / non-fraud)
- Subgraph detection
- Cycle detection

Industry Examples

- Banking fraud
- Insurance fraud
- Money laundering detection

3. Social Network Analysis:

Graph Modeling

- Nodes: Users
- Edges: Follow, friend, message

ML Tasks

- Community detection
- Influence maximization
- Link prediction

Algorithms

- PageRank
- Louvain clustering
- Graph embeddings

Examples

- Facebook friend suggestions
- Twitter (X) influencer ranking
- LinkedIn connection recommendations

4. Search Engines & Ranking Systems:

Graph Modeling

- Nodes: Web pages
- Edges: Hyperlinks

ML Tasks

- Page ranking
- Authority detection

Algorithms

- PageRank
- HITS
- GNN-based ranking

Real-World Example

- Google search ranking

5. Bioinformatics & Healthcare:

Graph Modeling

- Nodes: Proteins, genes, drugs
- Edges: Interactions, reactions

ML Tasks

- Graph classification
- Link prediction
- Node classification

Use Cases

- Drug discovery
- Protein interaction prediction
- Disease pathway analysis

6. Computer Vision:

Graph Modeling

- Nodes: Objects in image
- Edges: Spatial relationships

ML Tasks

- Scene graph generation
- Object interaction detection

Examples

- Autonomous driving
- Image captioning

7. NLP:

Graph Modeling

- Nodes: Words, entities, documents
- Edges: Co-occurrence, dependency

ML Tasks

- Document classification
- Entity linking
- Semantic similarity

Examples

- Knowledge-based QA
- Document clustering

Topics

Sunday, July 20, 2025

10:08 AM

1. Mathematical Foundations

- *Discrete Mathematics*
- *Recurrences & Growth Functions*
- *Probability & Expected Value*

2. Trees

- *Binary Search Tree*
- *Tree DP*
- *N-ary Tree*

3. Graphs

- *Graph Theory*
- *Traversal Algorithms*
- *Shortest Path Algorithms*
- *Topological Sorting & DAG Problems*
- *Disjoint Set Union*

4. Heaps

- *Mathematical Foundation*
- *Implementation*

5. Priority Queues (PQ)

- *PQ Theory*
- *Graph & PQ Applications*

6. Tries

- *Foundations*
- *Advanced Problems*

7. Backtracking

- *Theory*
- *Classic Problems*
- *Advanced Problems & Optimizations*

8. Dynamic Programming (DP)

- *Mathematical Foundation*
- *1-dimensional DP*
- *2-dimensional DP*
- *String DP*
- *DP on Trees & Graphs*

9. Segment Trees & Fenwick Trees

- *Segment Tree Theory*
- *Fenwick Trees*

10. Advanced Algorithmic Patterns